



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Desarrollo de un Agente Conversacional basado en IA para la Gestión Burocrática en la Universidad de Sevilla

Realizado por
Kevin Amador Calzadilla

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
José Enrique Sánchez López y Pablo Reina Jiménez

En el departamento de
Lenguajes y Sistemas Informáticos

15 de abril de 2026

Índice general

I	Introducción y Conceptos	1
1.	Introducción	2
1.1.	Marco conceptual	2
1.2.	Motivación	2
1.3.	Objetivos	3
1.3.1.	Objetivo general	3
1.3.2.	Objetivos técnicos	3
1.4.	Alcance del proyecto	4
1.5.	Estructura del proyecto	5
2.	Estado del Arte	6
2.1.	Modelos de Lenguaje Grande (LLMs)	6
2.1.1.	Definición y evolución	6
2.2.	Generación Aumentada por Recuperación (RAG)	7
2.2.1.	Concepto y arquitectura general	7
2.2.2.	Embeddings y bases de datos vectoriales	8
2.2.3.	Estrategias de fragmentación (chunking)	8
2.3.	Agentes de Inteligencia Artificial	9
2.3.1.	De LLMs a sistemas agénticos	9
2.3.2.	Grafos de estado: LangGraph	9
2.4.	Chatbots institucionales: antecedentes	10
2.5.	Tecnologías clave del ecosistema	10
2.5.1.	Lenguaje y Frameworks base	10
2.5.2.	Orquestación y Lógica Agéntica	11
2.5.3.	Pinecone como base de datos vectorial	11
2.5.4.	Cohere: embeddings y reranking	11
2.5.5.	Groq: inferencia de baja latencia	11
2.5.6.	LlamaParse: procesamiento de documentos	11
2.5.7.	Encriptado de contraseñas	12
2.5.8.	Gestión de la base de datos relacional	12
2.5.9.	Arquitectura de despliegue	12

II	Metodología y Planificación	15
3.	Metodología de desarrollo	16
3.1.	Ciclo de vida del proyecto	16
3.2.	Metodología Scrum adaptada	16
3.2.1.	Roles del proyecto	17
3.2.2.	Artefactos utilizados	17
3.2.3.	Dinámica de trabajo y reuniones	17
3.3.	Herramientas de gestión y desarrollo	18
4.	Planificación y desviaciones	19
4.1.	Definición estimada de sprints	19
4.2.	Diagrama de Gantt	20
4.3.	Estimación de esfuerzo	21
4.4.	Análisis de riesgos	21
4.5.	Desviaciones sobre la planificación inicial	22
4.6.	Estimación de costes	23
4.6.1.	Costes de personal	23
4.6.2.	Costes de hardware y software	23
4.6.3.	Costes indirectos	24
4.6.4.	Coste total del proyecto	24
III	Análisis y Diseño	25
5.	Análisis de requisitos	26
5.1.	Requisitos funcionales	26
5.2.	Requisitos no funcionales	27
5.3.	Casos de uso	27
6.	Diseño de la arquitectura	29
6.1.	Arquitectura general del sistema	29
6.2.	Arquitectura del backend (FastAPI)	29
6.2.1.	Modelo de datos	29
6.2.2.	Endpoints de la API REST	31
6.2.3.	Políticas de origen cruzado (CORS)	31
6.2.4.	Autenticación, Autorización y Roles	32
6.2.5.	Modularización y Desacoplamiento (Capa CRUD y Servicios)	32
6.3.	Diseño del agente conversacional	33
6.3.1.	Grafo de estados (LangGraph)	33
6.3.2.	Nodo Estado Inicial	34

6.3.3.	Nodo Router: clasificación de intenciones	34
6.3.4.	Nodo Recuperador: pipeline RAG	34
6.3.5.	Nodo Búsqueda Web: recuperación externa fallback	35
6.3.6.	Nodo Entrevistador: gestión de ambigüedades	35
6.3.7.	Nodos Resolutores: respuestas especializadas	35
6.3.8.	Nodo Rechazo Amable	36
6.4.	Diseño del sistema Dual RAG	36
6.5.	Diseño del frontend (Next.js + TypeScript)	36

IV Implementación 38

7. Infraestructura y despliegue 39

7.1.	Contenedorización con Docker	39
7.1.1.	Docker Compose: orquestación de servicios	39
7.1.2.	Configuración de red y variables de entorno	39
7.1.3.	Adaptación para despliegue y optimización en producción	39
7.2.	Base de datos PostgreSQL	40

8. Pipeline de ingesta de documentos 42

8.1.	Procesamiento con LlamaParse	42
8.2.	Fragmentación semántica (chunking)	42
8.2.1.	Evolución de la estrategia de fragmentación	42
8.2.2.	El problema del doble splitter	43
8.2.3.	Configuración final: MarkdownTextSplitter	43
8.3.	Generación de embeddings con Cohere	43
8.4.	Almacenamiento en Pinecone	44

9. Implementación del agente 45

9.1.	Evolución de la estructura del agente (grafo de estados)	45
9.1.1.	Primera aproximación: Pipeline RAG lineal (Versión 1)	45
9.1.2.	Segunda aproximación: Incorporación de control y enrutamiento (Versión 2)	46
9.1.3.	Tercera aproximación: Recuperación condicional (Versión 3)	47
9.2.	Esquema de estado (StateSchema)	48
9.3.	Sistema de prompts	49
9.3.1.	Consolidación del Análisis Inicial con Salida Estructurada	49
9.3.2.	Prompt de análisis inicial estructurado	49
9.3.3.	Prompt de suficiencia de información	50
9.3.4.	Prompts de los nodos resolutores	50

9.4.	Pipeline de recuperación	50
9.4.1.	Búsqueda Híbrida (Dense + Sparse) y RRF	50
9.4.2.	Reranking y Compresión con Cohere	51
9.5.	Gestión de memoria conversacional	51
9.5.1.	Ventana deslizante de mensajes	51
9.5.2.	Compresión de memoria con resumen	51
9.6.	Generación automática de títulos	52
9.7.	Corrective RAG (CRAG) y Búsqueda Web Fallback	52
9.8.	Perfilado Dinámico y Memoria Persistente de Usuario	53
9.8.1.	Prevención de saturación del contexto en el perfilado	53
10.	Implementación del frontend	55
10.1.	Arquitectura del Cliente (Next.js y React)	55
10.2.	Autenticación y Persistencia con JWT	56
10.3.	Interfaz del Chat y Consumo de SSE	56
10.3.1.	Resolución dinámica de la conexión backend	56
10.3.2.	Tratamiento amigable de fallos y límites de API (SSE)	57
10.4.	Diseño del Feedback Loop en Interfaz	57
10.5.	Panel de Auditoría y Administración	58
10.6.	Implementación de Modo Claro y Oscuro Adaptativo	58
11.	Selección y configuración de modelos	60
11.1.	Evolución de la infraestructura de inferencia	60
11.1.1.	Fase 1: Modelos locales (Llama 3.1 8B)	60
11.1.2.	Fase 2: API de HuggingFace	60
11.1.3.	Fase 3: API de Groq	60
11.2.	Estrategia multi-modelo	61
11.2.1.	LLM clasificador (Llama 3.1 8B Instant)	61
11.2.2.	LLM ligero (Llama 3.1 8B Instant)	61
11.2.3.	LLM principal (Llama 3.3 70B Versatile)	61
11.3.	Ajuste de hiperparámetros	61
V	Evaluación y Validación	62
12.	Estrategia de evaluación	63
12.1.	Dataset de evaluación	63
12.2.	Métricas empleadas	63
12.2.1.	Routing Accuracy	63
12.2.2.	Faithfulness	64

12.2.3. Answer Relevancy	64
12.2.4. Correctness	64
12.3. Evaluación con LLM como juez	64
12.4. Observabilidad y Monitoreo con Langfuse	64
12.5. Evaluación científica offline con Ragas	65
VI Conclusiones y Trabajo Futuro	66
13. Conclusiones	67
13.1. Cumplimiento de objetivos	67
13.2. Lecciones aprendidas	67
13.3. Dificultades encontradas	68
14. Trabajo futuro	70
14.1. Mejoras en el pipeline RAG	70
14.2. Ampliación de la base de conocimiento	70
14.3. Integración con canales de mensajería	70
14.4. Despliegue en producción	70
A. Prompts del sistema	73
B. Uso de la Inteligencia Artificial	81

Índice de figuras

2.1. Estructura típica de un sistema RAG	8
4.1. Diagrama de Gantt de la planificación estimada del TFG	21
6.1. Modelo de datos del sistema	30
6.2. Versión final diagrama de estados	34
9.1. Primera versión del flujo del agente (RAG lineal básico)	46
9.2. Segunda versión del flujo con enrutamiento posterior a la recuperación . .	47
9.3. Tercera versión con recuperación condicionada a la relevancia de la consulta	48

Índice de tablas

4.1. Planificación inicial estimada de sprints del proyecto	20
4.2. Matriz de riesgos del proyecto y sus estrategias de mitigación	22
4.3. Presupuesto total estimado	24
5.1. Requisitos funcionales del sistema	26
5.2. Requisitos no funcionales del sistema	27
6.1. Endpoints principales de la API	31
12.1. Distribución del dataset de evaluación por categoría	63

Parte I

Introducción y Conceptos

Capítulo 1

Introducción

1.1. Marco conceptual

En los últimos años, los Modelos de Lenguaje Grandes (LLMs) han revolucionado la forma en que las máquinas entienden y generan texto. Sin embargo, cualquiera que haya trabajado con ellos sabe que tienen un grave problema, las llamadas “alucinaciones”. Básicamente, el modelo puede generar respuestas que suenan perfectamente razonables pero que, en realidad, se las ha inventado por completo. Esto los convierte en herramientas poco fiables cuando necesitamos precisión, algo imprescindible si hablamos de la gestión de documentación universitaria.

Para lidiar con este problema, en este trabajo opté por implementar un sistema RAG (*Retrieval-Augmented Generation*). La idea es sencilla pero eficaz: en lugar de pedirle al modelo que “recuerde” la información, le doy acceso directo a la documentación oficial, previamente procesada y convertida en vectores. De esta manera, cada respuesta que genera combina la fluidez conversacional propia de un LLM con datos reales extraídos de fuentes institucionales verificables.

Nota: Estos conceptos y tecnologías base se definirán en profundidad en el Capítulo 2.

1.2. Motivación

Creo que cualquier persona que haya pasado por una universidad grande sabe lo desesperante que puede llegar a ser encontrar un dato concreto entre la gran cantidad de documentación institucional. En mi caso, como estudiante de la Universidad de Sevilla, he vivido esta situación que tanto me frustra más veces de las que me gustaría admitir: buscar un plazo de matrícula que aparece en un PDF de 80 páginas, intentar descifrar los criterios de un baremo de contratación o simplemente averiguar a qué ventanilla acudir para resolver un trámite. Y no soy el único: compañeros, profesores e incluso el propio personal administrativo se enfrentan a diario al mismo problema.

Lo habitual cuando surge una duda administrativa es descargar varios PDFs extensos y poco manejables, preguntar en la secretaría de la facultad —si es que coincide con el horario de atención— o enviar un correo electrónico sabiendo que la respuesta puede tardar días. Este desgaste no lo sufre solo el usuario, que puede perder horas buscando algo que debería encontrar en minutos; también lo padece el Personal de Administración y Servicios (PAS), cuya jornada se consume en buena parte respondiendo las mismas dudas una y otra vez, dudas que teóricamente ya están resueltas en los documentos publicados.

Ahí es precisamente donde nace la motivación de este proyecto: contar con un asistente virtual disponible las 24 horas, capaz de resolver consultas de forma inmediata, precisa y fundamentada en la documentación real de la universidad. A través de una interfaz sencilla, el sistema interpreta lo que el usuario necesita en lenguaje natural y le devuelve una respuesta clara, sin obligarle a bucear entre decenas de documentos.

1.3. Objetivos

El propósito de este proyecto es, en esencia, cambiar por completo la forma en que los usuarios interactúan con la densa normativa de la Universidad de Sevilla, apoyándonos para ello en tecnologías punteras del campo de la Inteligencia Artificial.

1.3.1. Objetivo general

Lo que busco con este trabajo es diseñar, desarrollar y poner a prueba un asistente conversacional inteligente basado en arquitecturas RAG (*Retrieval-Augmented Generation*). Este asistente debe ser capaz de ayudar a candidatos, estudiantes y docentes apoyándose únicamente en los documentos que los administradores le proporcionan. La clave está en que pueda interpretar consultas complejas sobre trámites burocráticos y normativos, y devolver respuestas lo suficientemente precisas como para que el usuario no tenga que recurrir a otras fuentes.

1.3.2. Objetivos técnicos

Para poder alcanzar ese objetivo general, me he propuesto una serie de hitos técnicos concretos:

- **Optimización de la ingesta de documentos:** Desarrollar un flujo de procesamiento mediante el que es posible interpretar documentos con estructuras complejas (tablas, anexos, notas al pie) mediante LlamaParse, garantizando que la información no pierda sentido al ser vectorizada.
- **Estructura agéntica:** Implementar un sistema basado en LangGraph con nodos especializados para la resolución de consultas:

- Nodos resultadores expertos para respuestas directas.
 - Nodo entrevistador para resolución de ambigüedades.
 - Nodo de rechazo para consultas fuera de dominio.
- **Diseño de un sistema dual RAG:** Construir una arquitectura capaz de separar el conocimiento normativo de las directrices de estilo y buenas prácticas cuando se atiende a un usuario.
 - **Minimizar el tiempo de respuesta:** Reducir los tiempos de respuesta a menos de veinte segundos combinando el uso de modelos pesados para generar la respuesta final con modelos livianos para las decisiones intermedias.
 - **Validación del sistema:** Evaluar el sistema con varios casos de uso reales que simulen conversaciones con estudiantes, docentes y personal administrativo. Los criterios de evaluación se basarán en métricas de *Faithfulness*, *Relevancy* y *Correctness*.

1.4. Alcance del proyecto

Conviene dejar claro desde el principio qué abarca este proyecto y qué queda fuera. En concreto, el trabajo incluye:

- El desarrollo de un **backend API REST** capaz de gestionar las peticiones de usuarios, el historial de conversaciones y toda la lógica del agente.
- La implementación de un **agente conversacional** basado en LangGraph con capacidad de enrutamiento inteligente y recuperación semántica de documentos.
- Un **pipeline de ingesta** que procesa documentos PDF oficiales de la US y los almacena como vectores en Pinecone.
- Una **interfaz web** moderna e interactiva desarrollada con Next.js (React) y TypeScript, estilizada con TailwindCSS y optimizada para una navegación fluida.
- La **contenedorización** completa del sistema mediante Docker Compose.
- Una **evaluación cuantitativa** del sistema.
- **Despliegue** en un entorno de producción con alta disponibilidad, como lo es Render

Queda fuera del alcance de este TFG la integración con sistemas internos de la US (como SEVIUS o la Secretaría Virtual), centrándome en la lógica del asistente, la personalización del usuario y el sistema integrado de feedback.

1.5. Estructura del proyecto

Para que la lectura de este documento resulte lo más fluida posible, lo he organizado en seis bloques temáticos bien diferenciados:

- **Parte I: Introducción y Conceptos.** Se expone la motivación del trabajo, los objetivos perseguidos y se realiza un repaso por el estado del arte de las tecnologías implicadas, incluyendo los LLMs, sistemas RAG y arquitecturas agénticas.
- **Parte II: Metodología y Planificación.** Describe el ciclo de vida elegido para el proyecto, detallando la adaptación del marco de trabajo Scrum y proporcionando el desglose temporal de las tareas.
- **Parte III: Análisis y Diseño.** Desglosa los requisitos funcionales y no funcionales, e introduce la arquitectura del sistema tanto a nivel conceptual (LangGraph) como de servicios (FastAPI, Next.js (React) + TypeScript, bases de datos).
- **Parte IV: Implementación.** Profundiza en los detalles técnicos del desarrollo: cómo se resolvieron los problemas de ingesta de documentos, la orquestación del agente y los desafíos surgidos con los LLM y su latencia.
- **Parte V: Evaluación y Validación.** Muestra los resultados obtenidos tras someter al agente a una exhaustiva batería de pruebas, utilizando un LLM como juez para evaluar su rigor y fiabilidad.
- **Parte VI: Conclusiones y Trabajo Futuro.** Recopila las reflexiones finales, valorando si se cumplieron los objetivos iniciales, y plantea posibles vías para continuar mejorando el sistema.

Capítulo 2

Estado del Arte

2.1. Modelos de Lenguaje Grande (LLMs)

2.1.1. Definición y evolución

Un Modelo de Lenguaje de Gran Escala (LLM, por sus siglas en inglés) es un sistema de *Deep Learning* fundamentado en redes neuronales con miles de millones de parámetros. Su entrenamiento principal consiste en asimilar cantidades masivas de texto sin etiquetar para aprender a predecir, estadísticamente, cuál es la siguiente palabra en una secuencia. El verdadero salto cualitativo de estos sistemas llegó con la introducción del mecanismo de *atención*, presentado en el influyente artículo 'Attention is all you need' [12]. Esta arquitectura, conocida como *Transformer*, permite al modelo sopesar qué partes del texto de entrada son más relevantes, procesando la información en paralelo y capturando el contexto a largo plazo de forma mucho más eficaz que sus predecesores.

Si echamos la vista atrás, desde los primeros modelos como GPT-2 en 2019 hasta las iteraciones actuales como Llama 3 [14], GPT-4 o Claude, el tamaño de estas redes ha crecido de forma desmesurada. Curiosamente, este aumento exponencial de parámetros ha revelado un fenómeno fascinante: al alcanzar cierta escala, los modelos desarrollan habilidades emergentes que nadie programó explícitamente durante su entrenamiento, tales como el razonamiento deductivo, la escritura de código o la capacidad de seguir instrucciones complejas.

Dicho todo esto, y a pesar de lo impresionantes que resultan, los LLMs arrastran limitaciones que conviene tener muy presentes:

- **Alucinaciones:** Quizá el problema más conocido. El modelo puede generar información que suena perfectamente creíble pero que es sencillamente falsa. Al fin y al cabo, lo que hace es predecir el siguiente token en función de patrones estadísticos; no tiene forma de “saber” si lo que dice es cierto o no.
- **Conocimiento congelado:** Todo lo que un LLM sabe quedó fijado en el momento

de su entrenamiento. No puede acceder por su cuenta a información nueva ni consultar documentos específicos de un ámbito concreto, a menos que alguien se los proporcione explícitamente.

- **Ventana de contexto limitada:** Aunque las ventanas de contexto han ido creciendo año tras año, el espacio del que dispone el modelo para “leer” información dentro de un prompt sigue siendo finito. Esto obliga a ser selectivos con qué le pasamos y cómo.

Estas carencias son las que han empujado a la comunidad investigadora a buscar técnicas complementarias que las mitiguen. En este trabajo nos centraremos en dos de ellas: los sistemas de Generación Aumentada por Recuperación (RAG) y las arquitecturas de agentes conversacionales.

2.2. Generación Aumentada por Recuperación (RAG)

2.2.1. Concepto y arquitectura general

La Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) es un paradigma propuesto por Lewis et al. [13] que combina la capacidad generativa de los LLMs con un sistema de recuperación de información. En lugar de confiar exclusivamente en el conocimiento paramétrico del modelo, RAG inyecta fragmentos de documentos relevantes en el prompt antes de la generación, permitiendo que la respuesta se fundamente en fuentes verificables.

El flujo típico de un sistema RAG consta de tres fases:

1. **Ingesta:** Los documentos se procesan, fragmentan y transforman en representaciones vectoriales (embeddings) que se almacenan en una base de datos vectorial.
2. **Recuperación:** Ante una consulta del usuario, se genera el embedding de la pregunta y se buscan los fragmentos más similares semánticamente en la base de datos.
3. **Generación:** Los fragmentos recuperados se inyectan como contexto en el prompt del LLM, que genera una respuesta basada en dicha información.

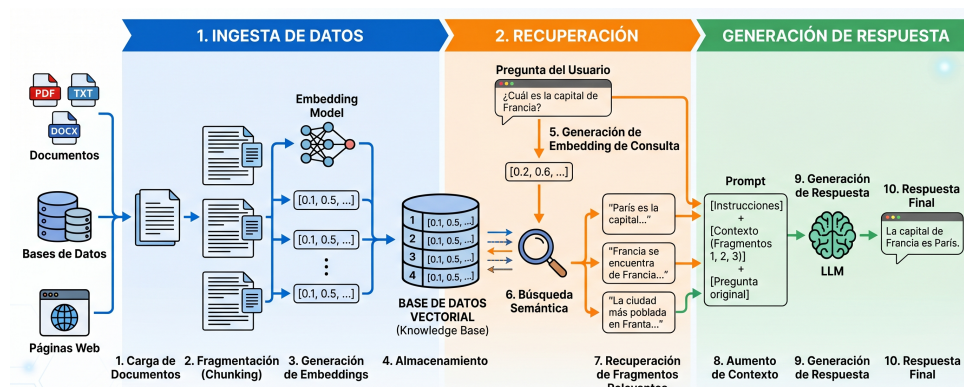


Figura 2.1: Estructura típica de un sistema RAG

2.2.2. Embeddings y bases de datos vectoriales

Un *embedding* es una representación numérica densa de un fragmento de texto en un espacio vectorial de alta dimensionalidad. Dos textos semánticamente similares producirán vectores cercanos en este espacio, lo que permite realizar búsquedas por similitud semántica en lugar de por coincidencia exacta de palabras.

Los modelos de embeddings multilingües son capaces de generar representaciones que capturan el significado del texto independientemente del idioma, lo que resulta esencial para un sistema que opera sobre documentación en español.

Las bases de datos vectoriales, como Pinecone, están optimizadas para almacenar y buscar eficientemente entre millones de vectores. Estas utilizan algoritmos de búsqueda aproximada de vecinos más cercanos (ANN, *Approximate Nearest Neighbors*) que permiten recuperar los documentos más relevantes en milisegundos.

2.2.3. Estrategias de fragmentación (chunking)

La fragmentación o *chunking* es el proceso de dividir documentos extensos en segmentos manejables para su vectorización. La calidad de esta fragmentación impacta directamente en la precisión de la recuperación, y existen diversos tipos como son los siguientes:

- **Fragmentación por tamaño fijo:** Divide el texto en bloques de n caracteres con un solapamiento (*overlap*) configurable. Es simple pero puede romper el contexto de tablas o párrafos.
- **Fragmentación semántica:** Respeta los límites naturales del documento (encabezados, párrafos, tablas) para preservar la coherencia. Herramientas como `MarkdownTextSplitter` de LangChain utilizan la estructura Markdown como guía para los cortes.
- **Fragmentación asistida por IA:** Herramientas como LlamaParse utilizan modelos de visión para interpretar la estructura del documento, preservando tablas, notas al pie y jerarquías de encabezados.

2.3. Agentes de Inteligencia Artificial

2.3.1. De LLMs a sistemas agénticos

Si me paro a pensarlo, un LLM por sí solo funciona como una caja que recibe texto y devuelve texto. Para lo que busco en este proyecto —un asistente que sea capaz de tomar decisiones, consultar fuentes y mantener el hilo de una conversación— eso se queda corto. Necesito un paso más: convertir ese modelo en el “cerebro” de un agente de IA que, además de generar lenguaje, pueda hacer lo siguiente:

- **Razonar:** Analizar la consulta del usuario y decidir las acciones que debe tomar.
- **Planificar:** Descomponer tareas complejas en subtareas para poder resolverlas de manera más eficiente.
- **Actuar:** Invocar herramientas externas, mediante las que enriquecer la calidad de la información a la que el modelo puede acceder (APIs, bases de datos, buscadores, etc).
- **Recordar:** Mantener un estado conversacional a lo largo de múltiples interacciones con el usuario.

2.3.2. Grafos de estado: LangGraph

Una vez que decidí trabajar con agentes, me hacía falta una herramienta que me permitiera orquestar todo esto desde Python. Después de evaluar varias alternativas, opté por utilizar LangGraph, una extensión de LangChain que modela el flujo de un agente como un grafo dirigido de estados. Cada operación se representa como un nodo del grafo, y las transiciones condicionales entre operaciones se definen como aristas. Este planteamiento ofrece varias ventajas claras frente al esquema clásico de entrada/salida:

- **Flujos condicionales:** En función de la entrada del usuario, el agente podrá tomar caminos diferentes teniendo a su disposición diversas herramientas.
- **Ciclos controlados:** Permite implementar bucles controlados de interacción con el usuario en los cuales el agente pueda solicitar información adicional en caso de ser algo ambigua la consulta.
- **Estado compartido:** Todos los nodos acceden y modifican a un estado global tipado (`StateSchema`), garantizando la coherencia.
- **Depuración:** Debido a la estructura de grafo facilita la trazabilidad de las decisiones del agente, facilitando de esta manera la identificación de errores.

2.4. Chatbots institucionales: antecedentes

La idea de poner un chatbot en una universidad no es nada nueva. Desde hace años, muchas instituciones han desplegado asistentes virtuales apoyándose en motores de Reconocimiento del Lenguaje Natural (NLU) y clasificación de intenciones, con plataformas conocidas como DialogFlow o Rasa. Estas herramientas funcionan razonablemente bien cuando el flujo de diálogo está bien acotado, pero se quedan cortas en cuanto el volumen de información crece. El motivo es que se basan en árboles de decisión y reglas predefinidas, lo que obliga a un mantenimiento constante: cada vez que surge una nueva casuística hay que añadir frases de entrenamiento y mapear la interacción manualmente.

Lo que diferencia a este trabajo de esos enfoques clásicos es el salto de un modelo basado en intenciones a uno generativo y dinámico. Combinando la flexibilidad de los LLMs con el conocimiento institucional actualizado mediante RAG y una estructura agéntica con distintos nodos especializados, el sistema es capaz de localizar la respuesta en la documentación oficial y presentarla de forma clara, sin necesidad de haber previsto cada posible pregunta de antemano.

2.5. Tecnologías clave del ecosistema

2.5.1. Lenguaje y Frameworks base

Como lenguaje principal utilicé Python. Tomé esta decisión de manera muy natural, dado que a día de hoy se considera que es el estándar en todo lo que rodea a la Inteligencia Artificial. Su ecosistema de bibliotecas es enorme y ofrece compatibilidad directa con las herramientas de orquestación que veré a continuación (LangChain, LangGraph), además de con prácticamente cualquier librería de procesamiento de datos que necesite.

Sobre esta base, estructuré el sistema utilizando dos *frameworks* principales, cada uno orientado a una de las capas de la aplicación (frontend/backend):

- **FastAPI (Backend):** para la implementación de la lógica del servidor y la exposición de servicios mediante una API REST y streaming por Server-Sent Events (SSE). Esta tecnología destaca por su alto rendimiento y su soporte nativo ante operaciones asíncronas, característica crítica para gestionar de manera eficiente las llamadas a los modelos de lenguaje y a la base de datos vectorial sin bloquear el hilo principal.
- **Next.js y TypeScript (Frontend):** para la interfaz de usuario. Aunque inicialmente me planteé usar Streamlit por su rapidez de prototipado, pronto requerí un nivel superior de interactividad, modularidad y control estético. La transición a una aplicación desarrollada en Next.js (basado en React) con TypeScript y TailwindCSS

me permitió separar limpiamente el cliente del servidor, implementar controles avanzados de sesión por cookies JWT, mostrar estados intermedios del agente a través de eventos en tiempo real, e integrar un panel de feedback y auditoría interactivo sin recargas invasivas de página.

2.5.2. Orquestación y Lógica Agéntica

LangChain [2] es un framework de código abierto que proporciona abstracciones para conectar LLMs con fuentes de datos externas, cadenas de procesamiento y herramientas. LangGraph [3] extiende este framework con la capacidad de definir grafos de estado, permitiendo flujos de control complejos con nodos y aristas condicionales.

2.5.3. Pinecone como base de datos vectorial

Pinecone [5] es un servicio gestionado de base de datos vectorial que ofrece almacenamiento, indexación y búsqueda de vectores de alta dimensionalidad con baja latencia. Su modelo serverless elimina la necesidad de gestionar infraestructura, lo que simplifica el despliegue.

2.5.4. Cohere: embeddings y reranking

Cohere [6] proporciona dos servicios fundamentales para este proyecto: un modelo de embeddings multilingüe (`embed-multilingual-v3.0`) para la vectorización de documentos en español, y un modelo de reranking (`rerank-v4.0-pro`) que reordena los resultados de la búsqueda vectorial según su relevancia contextual con la consulta.

2.5.5. Groq: inferencia de baja latencia

Groq [7] es una plataforma de inferencia que utiliza hardware especializado (LPUs, *Language Processing Units*) diseñado específicamente para la ejecución de modelos de lenguaje. Frente a las GPUs tradicionales, los LPUs ofrecen latencias de inferencia significativamente menores, lo que resulta crítico para un asistente conversacional en tiempo real.

2.5.6. LlamaParse: procesamiento de documentos

LlamaParse [8] es un servicio de procesamiento de documentos que utiliza modelos de visión por computador para extraer contenido de archivos PDF preservando su estructura original. A diferencia de extractores de texto convencionales, LlamaParse es capaz de interpretar tablas complejas, notas al pie y jerarquías de encabezados, convirtiendo el contenido a formato Markdown estructurado.

2.5.7. Encriptado de contraseñas

Para garantizar la seguridad y privacidad de los usuarios, implementé un flujo de *hashing* para las contraseñas utilizando el algoritmo bcrypt. Lo elegí principalmente porque, además de ofrecer un cifrado unidireccional (irrecuperable), incorpora de forma nativa un sistema de *salting*. Este consiste en añadir datos aleatorios a la contraseña antes de cifrarla, lo que permite ajustar el 'coste' computacional del algoritmo. Esto hace que el sistema sea resistente a ataques de fuerza bruta, garantizando que, incluso ante una filtración de la base de datos, los atacantes no puedan obtener las contraseñas reales.

2.5.8. Gestión de la base de datos relacional

Para la persistencia de los datos, me planteé diversas opciones, tales como MySQL, SQLServer o Oracle. Para el objetivo de este proyecto, consideré que la opción más conveniente era PostgreSQL, puesto que no solo ofrece una buena solución, sino que además ya tenía algo de experiencia previa trabajando con la misma. Se trata de un sistema de gestión de base de datos relacional que se caracteriza por:

- **Integridad y fiabilidad de datos:** garantiza el cumplimiento estricto de las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), lo que le permite ser muy tolerante a fallos, garantizando así transacciones muy seguras.
- **Código abierto y gratuito:** a diferencia de las bases de datos que requieren de una licencia comercial para poder hacer uso de todas sus funciones, esta tecnología al ser *open source* no tiene ningún tipo de coste, convirtiéndola en una alternativa rentable frente a opciones privativas.
- **Rendimiento y escalabilidad:** está diseñado para manejar grandes volúmenes de datos y cargas de trabajo pesadas, lo que le permite ser altamente escalable y eficiente en consultas complejas.

Para poder interactuar con la base de datos desde el backend, aunque valoré el uso de *psycopg2*, la cual ofrece una gran velocidad a la hora de resolver tareas sencillas, requería de un ORM (Object-Relational Mapping) más robusto. Finalmente, me decidí por *SQLAlchemy* dado que ofrece tanto el propio ORM como un *SQL Expression Language*, facilitando enormemente la realización de consultas. Además, destaca por su facilidad para gestionar conexiones, migraciones y proteger el sistema frente a vulnerabilidades como la inyección SQL.

2.5.9. Arquitectura de despliegue

Para poner el sistema a disposición de los usuarios finales en un entorno de producción real, opté por una arquitectura de despliegue basada en la contenedorización y en servicios

de plataforma en la nube (*Platform as a Service* o PaaS). El objetivo principal de este planteamiento es simplificar la puesta en marcha, asegurar la escalabilidad del sistema y garantizar que la aplicación funcione en producción de la misma forma exacta en que lo hace en el entorno de desarrollo local.

Las dos tecnologías clave que hacen esto posible son **Docker** y **Render**:

- **Docker para la contenedorización:** En lugar de lidiar con las diferencias de configuración y dependencias entre servidores, cada componente del sistema (el *frontend* en Next.js (React) y el *backend* en FastAPI) se empaqueta en su propio contenedor Docker utilizando imágenes base estables (como `node:18-alpine` para el cliente frontend y `python:3.11-slim-bookworm` para el servidor API backend). Esto encapsula todo lo necesario para la ejecución, aislando las aplicaciones y eliminando por completo el clásico problema del “en mi máquina funcionaba”.
- **Render para la infraestructura en la nube:** Render es una plataforma en la nube que destaca por su facilidad de uso e integración continua con GitHub. Permite desplegar aplicaciones web a partir de un **Dockerfile** de forma automatizada: cada vez que se sube un cambio al repositorio de producción, Render descarga el código, construye la imagen de Docker y despliega la nueva versión sin tiempo de caída.

En el entorno de producción, la arquitectura del sistema se distribuye de manera modular en tres servicios independientes dentro de la nube de Render:

1. **Servicio de Backend (FastAPI):** Configurado como un *Web Service* que expone los endpoints de la API. Se construye utilizando el archivo **Dockerfile** de la carpeta `/backend` y gestiona toda la lógica del agente LangGraph.
2. **Servicio de Frontend (Next.js):** Configurado como otro *Web Service* público. Se compila a partir del **Dockerfile** de la carpeta `/frontend` y sirve la interfaz de usuario que consume la API del backend.
3. **Base de Datos Gestionada (PostgreSQL):** En lugar de ejecutar PostgreSQL dentro de un contenedor autogestionado en producción, se utiliza un servicio de base de datos gestionada provisto directamente por Render. Esto garantiza copias de seguridad automáticas, alta disponibilidad y un rendimiento óptimo sin añadir complejidad de administración.

Para conseguir que todos estos servicios funcionen de manera óptima, la configuración de la aplicación se gestiona enteramente mediante **variables de entorno**. Siguiendo las buenas prácticas del desarrollo de software moderno (*Twelve-Factor App*), no se almacena ningún tipo de credencial o clave secreta en el código fuente ni dentro de las imágenes Docker.

En su lugar, Render permite inyectar estas variables en tiempo de ejecución de manera segura a través de su panel de administración. De esta forma, el *backend* recibe las claves de acceso de los proveedores de servicios (`GROQ_API_KEY`, `COHERE_API_KEY`, `PINECONE_API_KEY` y `LLAMA_CLOUD_API_KEY`), la clave secreta de firmado (`SECRET_KEY`) y la dirección de conexión a la base de datos (`DATABASE_URL`). Por su parte, el *frontend* únicamente necesita conocer la URL pública de la API del backend (`API_URL`) para poder enviarle las peticiones de los usuarios. Esta separación no solo protege la seguridad de nuestras credenciales, sino que dota al sistema de una enorme flexibilidad para cambiar de entorno o actualizar *API keys* sin necesidad de recompilar una sola línea de código.

Parte II

Metodología y Planificación

Capítulo 3

Metodología de desarrollo

3.1. Ciclo de vida del proyecto

Desde el primer momento tuve claro que un enfoque en cascada no iba a funcionar para este proyecto. Al tratarse de un sistema donde las decisiones sobre modelos, estrategias de fragmentación o hiperparámetros dependen en gran medida de resultados empíricos —es decir, de probar y ver qué sale—, necesitaba un ciclo de vida que me permitiera equivocarme pronto y corregir rápido. Por eso se optó por un enfoque iterativo e incremental, en el que cada iteración produce un incremento funcional del sistema.

En la práctica, cada iteración ha seguido el ciclo: análisis → diseño → implementación → pruebas → retroalimentación. Esto me permitió ir refinando componentes críticos como el pipeline de ingesta o la configuración de los modelos de lenguaje a medida que aprendía de los errores.

3.2. Metodología Scrum adaptada

Aunque Scrum está pensado para equipos multidisciplinares, sus principios básicos de flexibilidad y entrega incremental encajaban muy bien con el carácter de este proyecto. Lógicamente, no tenía sentido realizar reuniones de sincronización diarias (las tradicionales *dailies*) al ser un único desarrollador. Por ello, decidí adaptar el marco de trabajo a un formato unipersonal, simplificando la gestión de tareas pero manteniendo la disciplina de sprints cortos de dos semanas y el enfoque en desarrollar incrementos de software que se pudieran probar.

La dinámica real consistió en estructurar el trabajo en torno a las tutorías periódicas con mis tutores del TFG. En cada reunión se actuaba de forma similar a una revisión de sprint: evaluábamos el progreso, detectábamos fallos de diseño y decidíamos las prioridades para el siguiente ciclo de desarrollo.

3.2.1. Roles del proyecto

Al tratarse de un TFG individual, los roles tradicionales de Scrum se adaptaron para reflejar la relación real entre el alumno y los directores:

- **Product Owner (Clientes):** Este papel fue asumido por mis tutores del TFG, José Enrique Sánchez López y Pablo Reina Jiménez. Como expertos en el dominio y directores del trabajo, se encargaban de priorizar qué funcionalidades eran prioritarias para el asistente, proponer la documentación de la US a ingestar y validar que las respuestas generadas por el agente conversacional fueran rigurosas y útiles.
- **Scrum Master y Equipo de Desarrollo:** Yo mismo asumí estas funciones. Como desarrollador, me encargaba de codificar el frontend, el backend y configurar el pipeline RAG. Como Scrum Master, mi labor consistía en autoorganizar el trabajo, mantener al día el backlog y buscar soluciones rápidas a los bloqueos técnicos que surgían durante las pruebas locales.

3.2.2. Artefactos utilizados

Para llevar un control real del progreso sin sobrecargar el desarrollo con burocracia innecesaria, utilicé tres artefactos adaptados:

- **Backlog del Producto:** Una lista dinámica con las funcionalidades y mejoras que queríamos para el asistente. Esta lista varió enormemente a lo largo del proyecto a medida que nos topábamos con limitaciones de hardware o problemas de precisión en las respuestas del modelo.
- **Backlog del Sprint:** Las tareas concretas seleccionadas para realizarse entre tutorías (por ejemplo, implementar la comunicación entre backend y base de datos relacional, o pulir la visualización del chat).
- **Incremento:** La versión ejecutable y probada del software al final de cada ciclo. El objetivo fue que el asistente tuviera siempre una base estable y funcional sobre la que añadir la siguiente mejora.

3.2.3. Dinámica de trabajo y reuniones

Las ceremonias clásicas de Scrum se simplificaron para adaptarse al contexto académico de tutorías:

- **Planificación del Sprint:** Al comienzo de cada ciclo de dos semanas, definía los objetivos técnicos concretos a desarrollar en base a las recomendaciones e ideas surgidas en la tutoría previa.

- **Revisión y Retrospectiva:** En cada reunión con los tutores presentaba el incremento de software y probábamos juntos el comportamiento del agente. Esto nos servía para identificar qué partes (como la extracción de tablas de los PDFs o el diseño de los nodos en LangGraph) no estaban funcionando bien y debían replantearse de cara al siguiente sprint.

3.3. Herramientas de gestión y desarrollo

Para dar soporte a esta metodología de trabajo y asegurar la reproducibilidad del entorno de desarrollo, utilicé las siguientes herramientas:

- **Control de versiones:** Git como herramienta local y GitHub como repositorio remoto, lo que me permitió mantener un historial ordenado de cambios y proteger el código de pérdidas accidentales.
- **Gestión de tareas:** En lugar de usar herramientas corporativas complejas, organicé el backlog a través de listas de tareas priorizadas y notas vinculadas directamente con las actas de reuniones de tutoría.
- **Entorno de desarrollo:** Visual Studio Code, configurado con extensiones específicas para Python (depuración y tipado), Docker (para el control de los contenedores locales) y LaTeX (para la redacción de la memoria).
- **Entorno de ejecución local:** Docker y Docker Compose, esenciales para levantar de manera consistente y en pocos segundos el backend, frontend y la base de datos PostgreSQL, asegurando la reproducibilidad del entorno en cualquier máquina.

Capítulo 4

Planificación y desviaciones

4.1. Definición estimada de sprints

Para estructurar las 20 semanas de duración estimadas para el desarrollo del TFG, se organizó el cronograma de trabajo en 10 sprints de dos semanas cada uno. La siguiente tabla describe los hitos principales que se plantearon en cada uno de estos bloques temporales:

Tabla 4.1: Planificación inicial estimada de sprints del proyecto

Sprint	Periodo	Hito principal objetivo
1	Semanas 1–2	Investigación del estado del arte, selección de APIs y requisitos.
2	Semanas 3–4	Pipeline de ingesta: extracción semántica de PDFs oficiales.
3	Semanas 5–6	Conexión del RAG básico: embeddings de Cohere y Pinecone.
4	Semanas 7–8	Diseño lógico y programación del grafo de estados en LangGraph.
5	Semanas 9–10	Desarrollo del backend FastAPI y diseño del modelo de datos.
6	Semanas 11–12	Creación de la interfaz modular de chat en Next.js(React).
7	Semanas 13–14	Migración a la API de Groq para la optimización de latencias.
8	Semanas 15–16	Contenedorización de todos los servicios locales con Docker Compose.
9	Semanas 17–18	Batería de pruebas de validación y evaluación cuantitativa.
10	Semanas 19–20	Redacción final de la memoria y preparación de la defensa.

4.2. Diagrama de Gantt

Para visualizar mejor la distribución temporal de las tareas y comprender el solapamiento de las fases de desarrollo con la documentación y las pruebas, elaboré un diagrama de Gantt.

Este diagrama (representado en la Figura 4.1) actúa como una estimación del cronograma lógico seguido. Aunque en el día a día el desarrollo real fue dinámico e interactivo debido al ensayo y error, el Gantt sirve como marco de referencia para entender los plazos teóricos del proyecto.

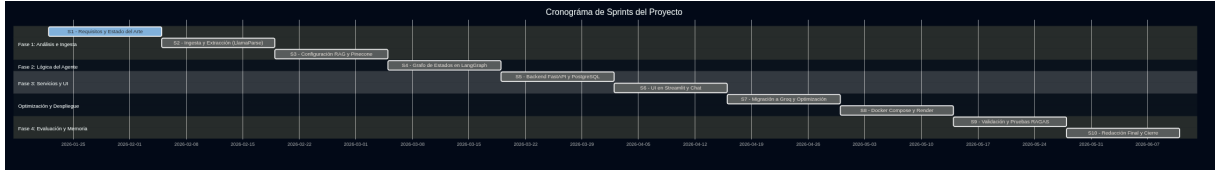


Figura 4.1: Diagrama de Gantt de la planificación estimada del TFG

4.3. Estimación de esfuerzo

Se planificó una dedicación media de entre 15 y 20 horas semanales durante las 20 semanas del proyecto, estimando un total acumulado de unas 350 horas de trabajo real. La distribución del tiempo según las distintas actividades del proyecto se desglosó de la siguiente forma aproximada:

- **Investigación teórica y estado del arte:** 15 % del tiempo (lectura de artículos sobre RAG, agentes conversacionales y documentación de APIs).
- **Desarrollo del pipeline RAG e ingesta:** 25 % del tiempo (pruebas de extracción de PDFs, vectorización y almacenamiento).
- **Orquestación del agente inteligente (LangGraph):** 20 % del tiempo (definición de nodos, transiciones condicionales y prompts de enrutamiento).
- **Backend y Frontend:** 15 % del tiempo (creación de la API de FastAPI, base de datos relacional e interfaz modular de chat en Next.js (React) con TypeScript).
- **Evaluación y validación:** 10 % del tiempo (baterías de pruebas para evaluar la fiabilidad de las respuestas).
- **Redacción de la memoria y documentación:** 15 % del tiempo (documentación técnica y redacción de este documento).

4.4. Análisis de riesgos

Para evitar que el desarrollo se estancara por problemas técnicos imprevistos, realicé un análisis de riesgos inicial evaluando su probabilidad e impacto y definiendo medidas paliativas concretas:

Tabla 4.2: Matriz de riesgos del proyecto y sus estrategias de mitigación

Riesgo identificado	Prob.	Imp.	Estrategia de mitigación planteada
Límites de cuota gratuitos en APIs (Groq, LlamaParse)	Alta	Alto	Diseñar un sistema multi-modelo usando LLMs ligeros.
Alucinaciones e invenciones en respuestas del agente	Media	Alto	Forzar temperatura baja (0.1) y RAG estricto con fuentes.
Pérdida de estructura en tablas complejas de los PDFs	Alta	Medio	Migración a LlamaParse con prompts específicos de extracción.
Latencias excesivas de inferencia en tiempo de chat	Media	Medio	Uso de hardware optimizado (LPUs de Groq) para inferencia rápida.
Pérdida de contexto en chats muy prolongados	Baja	Bajo	Implementación de memoria de ventana y compresión por resúmenes.
Curva de aprendizaje del framework LangGraph	Media	Medio	Reservar margen de tiempo en los primeros sprints de diseño.

4.5. Desviaciones sobre la planificación inicial

En un proyecto centrado en la integración de Inteligencia Artificial y modelos de lenguaje, la teoría rara vez coincide al 100 % con la práctica. La naturaleza exploratoria del desarrollo hizo que el cronograma real sufriera variaciones significativas respecto a la planificación estimada en el Gantt, destacando cuatro desviaciones principales:

- **Cambio en la infraestructura de inferencia (Sprints 4 y 7):** Inicialmente, la intención era que todo el sistema funcionara en local utilizando modelos de código abierto (como Llama 3.1 8B) a través de Ollama. Sin embargo, al probar las primeras ejecuciones del agente LangGraph, los tiempos de respuesta superaban con creces el minuto por consulta debido a la falta de potencia de hardware local. Esto obligó a cambiar el rumbo técnico a mitad del proyecto para migrar todas las llamadas de inferencia a la API en la nube de Groq.
- **Reestructuración del pipeline de ingesta (Sprints 2 y 3):** La fragmentación estándar de los documentos oficiales (normativas de matrícula o guías de viaje) rompía por completo las tablas y los anexos, lo que provocaba que el asistente diera

respuestas erróneas sobre plazos y cuantías de dinero. Tuve que detener el plan de desarrollo previsto para investigar tecnologías de extracción avanzada, migrando primero a Docling y finalmente a LlamaParse. El proceso de solucionar el problema del "doble splitter" (donde un formateador secundario rompía de nuevo las tablas de LlamaParse) consumió casi dos semanas completas que no estaban contempladas originalmente.

- **Gestión de cuotas y depuración de la memoria:** El testing del agente consumió rápidamente los límites de tokens diarios de las capas gratuitas de las APIs. Esto provocó parones forzados en el desarrollo y obligó a implementar a toda prisa la funcionalidad de compresión y resumen de memoria histórica mediante tareas en segundo plano en FastAPI, con el fin de ahorrar tokens en cada turno de chat.
- **Reconstrucción de la interfaz frontend (Sprints 5 y 6):** Inicialmente se contempló el uso de Streamlit para el desarrollo rápido del frontend. No obstante, al avanzar en los requisitos de interactividad, control estético personalizado y la necesidad de gestionar sesiones de usuario mediante cookies JWT, flujos Server-Sent Events (SSE) interactivos y un panel de administración avanzado, Streamlit demostró limitaciones de modularidad y escalabilidad. Esto forzó a pivotar la arquitectura del cliente hacia Next.js (React) con TypeScript. Reconstruir la interfaz desde cero supuso un esfuerzo extra de desarrollo y una curva de aprendizaje no prevista que desplazó ligeramente el calendario de integración.

4.6. Estimación de costes

Aunque este proyecto es puramente académico, resulta interesante hacer el ejercicio de estimar cuánto costaría si se tratase de un encargo profesional real desarrollado por un ingeniero de software junior. He dividido la estimación en tres partidas: personal, recursos materiales y costes indirectos.

4.6.1. Costes de personal

Tomando como referencia la estimación de 350 horas de dedicación calculada en la sección anterior, y asumiendo un coste por hora estándar para un perfil de desarrollador junior de 15 €/hora, el coste de personal asciende a:

- **Desarrollador Junior:** $350 \text{ horas} \times 15 \text{ €/hora} = 5.250 \text{ €}$

4.6.2. Costes de hardware y software

- **Hardware:** Emplé un equipo informático portátil valorado en 1.200 €. Considerando un periodo de amortización de 4 años, y dado que el proyecto duró 5 meses,

la cuota de amortización imputable al proyecto es de aproximadamente 125 €.

- **Software y APIs:** El desarrollo lo apoyé mayoritariamente en herramientas de código abierto (Python, Next.js (React), FastAPI, Docker). Los servicios de IA y bases de datos vectoriales (Groq, Cohere, Pinecone, LlamaParse) los utilicé en su capa gratuita (*Free Tier*), por lo que el coste directo en este apartado es de 0 €. Sin embargo, en un entorno de producción, los costes de inferencia y almacenamiento escalarían según el uso.

4.6.3. Costes indirectos

Se contemplan gastos de suministros como electricidad y conexión a internet durante los 5 meses de desarrollo, que se estiman conservadoramente en 20 € mensuales imputables al proyecto.

- **Suministros:** 5 meses $\times$ 20 €/mes = 100 €

4.6.4. Coste total del proyecto

Sumando todas las partidas anteriores, obtengo el coste total estimado del proyecto:

Tabla 4.3: Presupuesto total estimado

Concepto	Coste estimado (€)
Costes de Personal	5.250,00
Costes de Hardware (Amortización)	125,00
Costes de Software y APIs	0,00
Costes Indirectos	100,00
Base Imponible	5.475,00
IVA (21 %)	1.1149,75
Total Presupuesto	6.624,75

Parte III

Análisis y Diseño

Capítulo 5

Análisis de requisitos

5.1. Requisitos funcionales

Tabla 5.1: Requisitos funcionales del sistema

ID	Descripción
RF-01	El sistema debe permitir el registro de usuarios mediante email y contraseña.
RF-02	El sistema debe permitir la autenticación de usuarios mediante JWT.
RF-03	El usuario debe poder crear, renombrar y eliminar conversaciones.
RF-04	El sistema debe procesar la consulta del usuario y generar una respuesta fundamentada en documentos oficiales.
RF-05	El sistema debe clasificar la intención del usuario y enrutar la consulta al nodo especializado correspondiente.
RF-06	El sistema debe rechazar amablemente consultas que no estén relacionadas con trámites de la US.
RF-07	El sistema debe solicitar aclaraciones al usuario cuando la consulta sea ambigua.
RF-08	El sistema debe mostrar las fuentes y referencias consultadas junto con cada respuesta.
RF-09	El sistema debe mantener el contexto de la conversación a lo largo de múltiples turnos.
RF-10	Los administradores deben poder ingestar nuevos documentos PDF al sistema desde la interfaz.
RF-11	El sistema debe generar automáticamente títulos descriptivos para las conversaciones.

Tabla 5.1: Requisitos funcionales del sistema (continuación)

ID	Descripción
RF-12	El sistema debe comprimir la memoria de conversaciones largas para mantener la coherencia.
RF-13	El sistema debe mostrar a los administradores las cuentas de usuario existentes en el programa.
RF-14	El sistema debe permitir a los administradores eliminar cuentas de usuario.
RF-15	El sistema debe permitir a los administradores convertir a otros usuarios en administradores.
RF-16	El sistema debe permitir a los administradores degradar a otros administradores en usuarios.

5.2. Requisitos no funcionales

Tabla 5.2: Requisitos no funcionales del sistema

ID	Descripción
RNF-01	El tiempo de respuesta del agente no debe superar los 20 segundos.
RNF-02	El sistema debe desplegarse mediante contenedores Docker para garantizar la portabilidad.
RNF-03	Las contraseñas deben almacenarse hasheadas con bcrypt.
RNF-04	La interfaz debe ser accesible desde cualquier navegador web moderno.
RNF-05	El sistema debe ser capaz de procesar documentos PDF de hasta 100 páginas.
RNF-06	Las respuestas del agente deben basarse exclusivamente en los documentos ingestados (sin alucinaciones).
RNF-07	La arquitectura debe permitir la adición de nuevos tipos de nodos sin modificar el núcleo del sistema.

5.3. Casos de uso

Los principales casos de uso identificados son:

- **CU-01: Registrarse.** Un usuario nuevo crea una cuenta proporcionando email y contraseña.
- **CU-02: Iniciar sesión.** Un usuario registrado se autentica para acceder al sistema.
- **CU-03: Realizar consulta burocrática.** El usuario formula una pregunta sobre normativa o trámites de la US y recibe una respuesta fundamentada.
- **CU-04: Gestionar conversaciones.** El usuario crea, renombra o elimina sus conversaciones.
- **CU-05: Ingestar documento (Admin).** Un administrador sube un nuevo PDF para ampliar la base de conocimiento del agente.
- **CU-06: Gestionar usuarios (Admin).** Un administrador puede gestionar usuarios ya sea añadiendo como nuevo administrador, eliminándolo, degradándolo a usuario normal.

Capítulo 6

Diseño de la arquitectura

6.1. Arquitectura general del sistema

El sistema sigue una arquitectura de microservicios contenedorizados orquestados mediante Docker Compose. Los tres servicios principales son:

- **Frontend (Next.js + TypeScript):** Interfaz web que se comunica con el backend vía HTTP REST y Server-Sent Events (SSE). Puerto 8501.
- **Backend (FastAPI):** API REST que gestiona la autenticación, las conversaciones y orquesta el agente LangGraph. Puerto 8000.
- **Base de datos (PostgreSQL 15):** Almacena usuarios, conversaciones y mensajes. Puerto 5432.

Adicionalmente, el sistema se conecta con servicios externos en la nube:

- **Pinecone:** Base de datos vectorial para almacenamiento y búsqueda de embeddings.
- **Groq:** API de inferencia para los modelos Llama 3.1 8B y Llama 3.3 70B.
- **Cohere:** API para generación de embeddings y reranking de documentos.
- **LlamaParse:** API para el procesamiento inteligente de PDFs.

6.2. Arquitectura del backend (FastAPI)

6.2.1. Modelo de datos

El modelo relacional del sistema se compone de tres entidades principales implementadas con SQLAlchemy ORM:

- **Usuario:** Almacena el email, contraseña hasheada, rol de administrador y fecha de creación. Relación uno a muchos con Conversación.
- **Conversación:** Contiene el título (auto-generado), un campo de resumen de memoria para conversaciones largas y la referencia al usuario propietario. Relación uno a muchos con Mensaje.
- **Mensaje:** Almacena el rol (user/assistant), el contenido textual, las referencias a documentos consultados (en formato JSON) y la marca temporal.

Las relaciones incluyen eliminación en cascada: al eliminar un usuario se eliminan sus conversaciones, y al eliminar una conversación se eliminan sus mensajes.

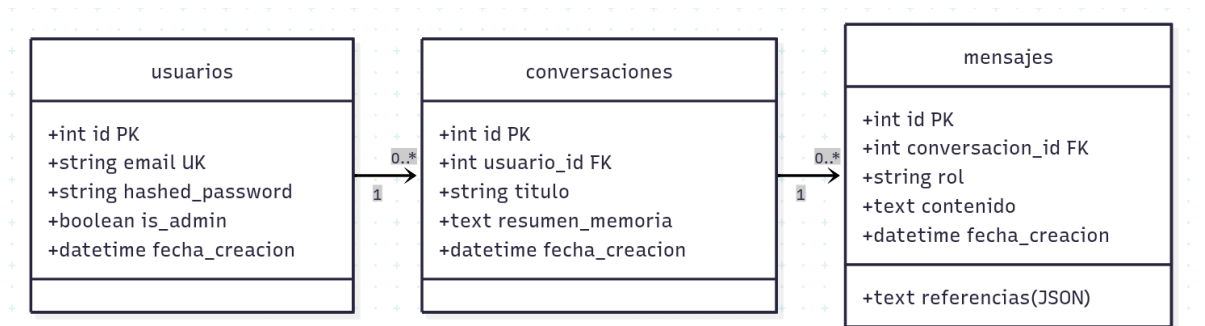


Figura 6.1: Modelo de datos del sistema

6.2.2. Endpoints de la API REST

Tabla 6.1: Endpoints principales de la API

Método	Ruta	Descripción
POST	/auth/registro	Registro de nuevo usuario
POST	/auth/login	Autenticación con email/contraseña
GET	/auth/me	Datos del usuario autenticado
GET	/conversaciones	Lista de conversaciones del usuario
POST	/conversaciones	Crear nueva conversación
GET	/conversaciones/{id}/ mensajes	Historial de mensajes
POST	/conversaciones/{id}/ chat	Enviar mensaje y obtener respuesta del agente
POST	/conversaciones/{id}/ mensajes/{mid}/feedback	Enviar valoración y comentario
PUT	/conversaciones/{id}	Renombrar conversación
DELETE	/conversaciones/{id}	Eliminar conversación
GET	/admin/feedback/negativo	Listar feedback negativo y referencias (Admin)
POST	/admin/ingestar	Subir e indexar archivo PDF (Admin)
DELETE	/admin/documentos	Borrar base de datos vectorial (Admin)
GET	/admin/usuarios	Listar cuentas de usuarios (Admin)
DELETE	/admin/usuarios/{id}	Eliminar cuenta de usuario (Admin)
PUT	/admin/usuarios/{id}/ admin	Conceder/revocar rol de admin (Admin)
GET	/documentos	Listar los nombres de los documentos en Pinecone

6.2.3. Políticas de origen cruzado (CORS)

Dado que el frontend (Next.js) y el backend (FastAPI) corren en puertos o subdominios independientes, los navegadores bloquean de forma nativa las peticiones HTTP cruzadas. Para solucionar esto, configuré el middleware de CORS en FastAPI (`CORSMiddleware`) para permitir el acceso de forma controlada a los métodos y cabeceras necesarios, garantizando que el chat y el streaming por SSE funcionen correctamente sin bloqueos del navegador.

6.2.4. Autenticación, Autorización y Roles

El sistema implementa un control de accesos basado en tokens JWT (*JSON Web Tokens*) firmado mediante el algoritmo simétrico HS256. Al iniciar sesión, el servidor valida las credenciales y devuelve el token, el cual expira a los 7 días. El cliente envía este token en la cabecera HTTP de autorización para cualquier operación.

A nivel de base de datos, implementé roles diferenciados mediante la columna booleana `is_admin` en la tabla de usuarios. Esto habilita dos niveles de permisos:

- **Rol de Usuario:** Permite registrarse, iniciar sesión, crear conversaciones de chat y enviar feedback sobre las respuestas.
- **Rol de Administrador:** Otorga acceso a todos los endpoints bajo la ruta raíz `/admin`. Esto habilita las funciones críticas de:
 1. **Gestión de Usuarios:** Listar todos los perfiles de usuario de la plataforma, eliminar cuentas y promover o degradar cuentas concediendo el rol de administrador.
 2. **Gestión de Documentos:** Ingestar nuevos PDFs con las normativas de la US o vaciar la base de datos vectorial de Pinecone mediante la llamada a `DELETE /admin/documentos`.
 3. **Auditoría de Feedback:** Inspeccionar las valoraciones negativas recibidas para corregir el RAG.

6.2.5. Modularización y Desacoplamiento (Capa CRUD y Servicios)

A medida que el backend crecía, me di cuenta de que mantener todo el código en un único archivo (`main.py`) era una mala idea. El archivo principal acumulaba más de 500 líneas y mezclaba demasiadas responsabilidades: la definición de endpoints, la inyección de dependencias de usuario, la lógica de acceso a la base de datos con SQLAlchemy y la orquestación del streaming SSE con LangGraph. Esto no solo dificultaba enormemente la lectura y la depuración del código, sino que ponía en riesgo la escalabilidad del sistema.

Para solucionar este problema de acoplamiento, decidí llevar a cabo una refactorización profunda estructurando el backend en una arquitectura modular por capas:

- **Capa de Controladores (Routers):** Dividí los endpoints de la API REST utilizando `APIRouter` de FastAPI en archivos independientes dentro de una nueva carpeta `/routers`: `auth.py` (para el registro, login y perfil), `conversaciones.py` (para la gestión de chats y mensajes), `admin.py` (para operaciones administrativas) y `documentos.py` (para consultas del RAG).

- **Capa CRUD/Acceso a Datos (`crud.py`):** Con el fin de aislar las consultas y modificaciones en la base de datos relacional, extraje todas las llamadas directas de SQLAlchemy (`db.query`, `db.commit`, etc.) fuera de los controladores. Aunque consideré llamarle a esta capa *Repository* de acuerdo con patrones formales como DDD (Domain-Driven Design), preferí mantener el término `crud.py` por ser la convención estándar recomendada en la documentación oficial de FastAPI.
- **Capa de Servicios de Negocio (`services/`):** Endpoints extremadamente complejos como la transmisión asíncrona por Server-Sent Events (SSE) en `enviar_mensaje` los simplifiqué abstrayendo sus responsabilidades. Creé un servicio de chat (`chat_service.py`) encargado de dar formato al historial reciente e interactuar con el agente en segundo plano, dejando el router del endpoint con apenas 20 líneas de código dedicadas a validar la petición HTTP y retornar la respuesta en streaming.

Esta separación de responsabilidades no solo incrementó drásticamente la legibilidad del proyecto, sino que me permitió solventar un bug latente de tipado en el formateador del stream del agente (al extraer claves con un iterador incorrecto en tiempo de ejecución) y facilitó la integración limpia de las dependencias comunes de autorización mediante tipados avanzados con `Annotated` de Python.

6.3. Diseño del agente conversacional

6.3.1. Grafo de estados (LangGraph)

Implementé el agente como un `StateGraph` de LangGraph con ocho nodos y transiciones condicionales. El estado compartido entre nodos se define mediante un `TypedDict` (`StateSchema`) con los siguientes campos: `pregunta`, `pregunta_reformulada`, `historial`, `historial_formateado`, `contexto`, `stream` y `referencias`.

El flujo general del grafo es:

1. **START** → **Estado Inicial**: Formatea el historial de la conversación.
2. **Estado Inicial** → **Recuperador** — **Rechazo**: Decide si la consulta es relevante para el dominio.
3. **Recuperador** → **Entrevistador** — **Búsqueda Web** — **Clasificador**: Decide si la información es suficiente (va al clasificador), ambigua (va al entrevistador) o insuficiente (va a la búsqueda web de fallback).
4. **Búsqueda Web** → **Clasificador**: Enriquece el contexto con la búsqueda externa y continúa hacia el clasificador.

5. **Clasificador** → **Procedimental** — **Calendario** — **Normativo** — **Baremo**:
Enruta la consulta al nodo resolutor experto adecuado.
6. **Nodos finales** → **END**: Genera la respuesta definitiva y finaliza el flujo.

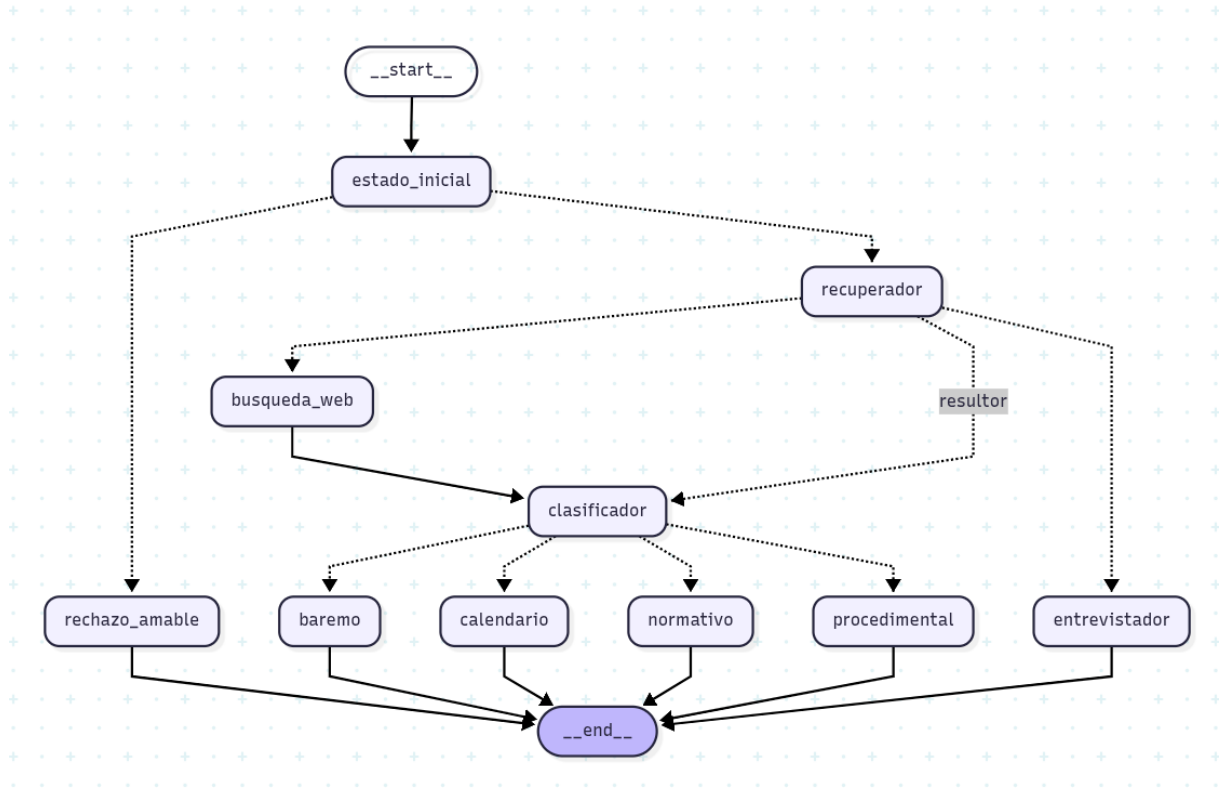


Figura 6.2: Versión final diagrama de estados

6.3.2. Nodo Estado Inicial

Formatea el historial de mensajes de la conversación en un formato legible (“rol: contenido”) que será utilizado por los prompts de los nodos posteriores.

6.3.3. Nodo Router: clasificación de intenciones

Utiliza un LLM clasificador ultra-ligero (Llama 3.1 8B con `max_tokens=15`) para determinar si la consulta del usuario es relevante para el dominio del agente. Devuelve “recuperador” si la consulta está relacionada con trámites de la US, o “rechazo_amable” en caso contrario.

6.3.4. Nodo Recuperador: pipeline RAG

Este nodo ejecuta el pipeline completo de recuperación de información:

1. Reformula la pregunta del usuario integrando el contexto del historial.
2. Genera el embedding de la pregunta reformulada con Cohere.
3. Busca los 12 documentos más similares en Pinecone.
4. Aplica reranking con Cohere (**rerank-v4.0-pro**) seleccionando los 4 más relevantes.
5. Formatea los documentos con metadata (fuente y página).

6.3.5. Nodo Búsqueda Web: recuperación externa fallback

Este nodo se activa cuando el evaluador de suficiencia determina que los documentos internos recuperados de la base de conocimiento vectorial en Pinecone son insuficientes para resolver la pregunta. En este caso, realizo una consulta a la API de DuckDuckGo filtrada estrictamente al dominio de la Universidad de Sevilla (**site:us.es**) para obtener información fresca de internet. Los extractos y referencias recuperados se anexan al contexto del grafo y la ejecución prosigue hacia el clasificador para responder de forma precisa.

6.3.6. Nodo Entrevistador: gestión de ambigüedades

Este nodo se activa cuando el sistema de suficiencia detecta que la consulta del usuario es ambigua o carece de parámetros clave para ofrecer una resolución definitiva (por ejemplo, al preguntar sobre "becas" de manera genérica). Para evitar que el agente formule preguntas de aclaración al vacío, decidí dotar a este nodo de una herramienta de búsqueda web en tiempo de ejecución (filtrada al dominio **site:us.es**). De esta forma, el entrevistador realiza una consulta complementaria, enriquece el contexto inicial obtenido de Pinecone y presenta al usuario un resumen orientador de las opciones generales detectadas antes de formular la pregunta aclaratoria de seguimiento. Adicionalmente, implementé una regla estricta en el grafo para impedir que el agente encadene dos preguntas aclaratorias consecutivas, evitando así cansar al usuario.

6.3.7. Nodos Resolutores: respuestas especializadas

Se han diseñado cuatro nodos resolutores, cada uno con un prompt especializado:

- **Procedimental:** Genera respuestas paso a paso para trámites administrativos (matrícula, liquidación de viajes, etc.), usando listas numeradas y resaltando plataformas y documentos requeridos.
- **Calendario:** Especializado en fechas, plazos y periodos del calendario académico, con especial énfasis en la precisión de las fechas.

- **Normativo:** Explica normativas, requisitos legales y resoluciones rectorales, citando artículos específicos cuando están disponibles en el contexto.
- **Baremo:** Detalla criterios de evaluación, puntuaciones y méritos en procesos de selección de profesorado, utilizando tablas cuando es apropiado.

6.3.8. Nodo Rechazo Amable

Genera una respuesta educada y empática indicando al usuario que su consulta no puede ser atendida, sugiriendo alternativas cuando es posible.

6.4. Diseño del sistema Dual RAG

El sistema implementa dos niveles de RAG paralelos:

- **RAG de conocimiento normativo:** La base de datos vectorial en Pinecone contiene los fragmentos de los documentos oficiales de la US (normativas de matrícula, baremos de contratación, calendarios académicos, guías de viajes, etc.).
- **RAG de buenas prácticas:** Los prompts de cada nodo resolutor incorporan directrices de estilo, formato y tono que guían la generación de respuestas cercanas al usuario sin sacrificar el rigor técnico.

De esta forma, el contenido proviene exclusivamente de los documentos oficiales, mientras que la forma y estructura de la respuesta se define mediante las instrucciones embebidas en los templates de cada nodo.

6.5. Diseño del frontend (Next.js + TypeScript)

La interfaz web se ha diseñado como una aplicación modular estructurada en cinco áreas principales:

- **Pantalla de autenticación:** Interfaz de acceso y registro interactiva. La sesión del usuario se mantiene de forma segura mediante una cookie JWT, lo que evita la necesidad de volver a autenticarse al recargar la página.
- **Barra lateral de navegación (Sidebar):** Contiene la lista histórica de conversaciones del usuario (con opciones rápidas para crear chats, renombrarlos o eliminarlos) y, si el usuario tiene privilegios de administrador, acceso a los paneles de gestión documental y auditoría de feedback.

- **Área de chat interactivo:** Área del chat que implementa animaciones de entrada, burbujas de conversación diferenciadas por colores y un renderizador de Markdown que visualiza negritas, listas y enlaces de manera legible.
- **Canal de visualización de estados del agente (SSE):** Indicadores de estado dinámicos que muestran el paso lógico por el que transiciona el agente LangGraph en el backend (por ejemplo, cuando evalúa documentos o realiza búsquedas externas).
- **Sistema de Feedback e Inline Comments:** Botones de feedback positivo y negativo integrados de manera discreta al final de cada respuesta del asistente. Al votar negativamente, se despliega una caja de texto simple para recopilar de forma directa el motivo del descontento del usuario.
- **Panel de Auditoría de Feedback (Admin):** Una vista dedicada que organiza de forma visual las quejas recopiladas, permitiendo que los administradores analicen qué pregunta originó el fallo, qué respondió el asistente, qué comentario dejó el usuario y qué fragmentos de Pinecone se consultaron.

Diseñé la interfaz visual con una estética adaptativa (con soporte para modo claro y oscuro) utilizando variables CSS semánticas integradas con TailwindCSS. Por defecto, la aplicación detecta el esquema de color preferido del dispositivo del usuario (a través de la consulta de medios `prefers-color-scheme`) para inicializar el tema correspondiente. Adicionalmente, agregué un selector manual en la barra lateral para que el usuario pueda alternar entre temas en caliente, guardando de forma persistente su preferencia en el `localStorage` del navegador. Las paletas de colores las seleccioné para garantizar altos ratios de contraste y legibilidad, manteniendo la identidad de la marca mediante transiciones suaves de micro-animaciones en botones, iconos y modales que hacen hover con el color granate corporativo de la Universidad de Sevilla (`#9D1C34`).

Parte IV

Implementación

Capítulo 7

Infraestructura y despliegue

7.1. Contenedorización con Docker

7.1.1. Docker Compose: orquestación de servicios

El sistema se despliega mediante un archivo `docker-compose.yml` que define tres servicios:

- **db:** Contenedor PostgreSQL 15 con volumen persistente (`postgres_data`) para la durabilidad de los datos. Se expone en el puerto 5434 del host.
- **backend:** Contenedor con la aplicación FastAPI, que se construye desde el directorio `./backend`. Depende del servicio `db` y se expone en el puerto 8000.
- **frontend:** Contenedor con la aplicación Next.js, construido desde el directorio `./frontend`. Se conecta al backend mediante la URL interna `http://backend:8000` (o resolviéndose dinámicamente en el cliente) y se expone en el puerto 8501 del host.

7.1.2. Configuración de red y variables de entorno

Docker Compose crea automáticamente una red interna que permite la comunicación entre servicios por nombre de host. Las credenciales de bases de datos, claves API (Groq, Pinecone, Cohere, LlamaParse) y parámetros de configuración se gestionan mediante un archivo `.env` compartido, evitando la exposición de secretos en el código fuente.

7.1.3. Adaptación para despliegue y optimización en producción

Durante la fase de puesta en producción (especialmente en la plataforma Render), surgieron dos desafíos técnicos de infraestructura que obligaron a refactorizar las directivas de construcción y ejecución de los contenedores:

1. **Puertos dinámicos en el backend:** Las plataformas de hosting en la nube no permiten mapear puertos fijos de forma externa, sino que asignan dinámicamente una variable de entorno llamada `PORT` a la instancia en tiempo de ejecución. Modifiqué la instrucción de arranque en el `Dockerfile` del backend, reemplazando el puerto estático por una evaluación de variable de entorno mediante Bash: `uvicorn main:app --host 0.0.0.0 --port ${PORT:-8000}`. De este modo, el contenedor arranca por defecto en el puerto 8000 a nivel local pero se amolda de forma transparente a cualquier puerto requerido por el proveedor en la nube.
2. **Reducción de consumo de memoria en el frontend:** En las primeras pruebas, el contenedor del frontend consumía excesivos recursos de memoria RAM (llegando a superar el límite de 512 MB de la capa gratuita y forzando reinicios por OOM, o *Out of Memory*). Para solucionarlo, reescribí el ciclo de construcción en su `Dockerfile`: en lugar de correr el servidor en caliente usando la herramienta de desarrollo (`next dev`), configuré una fase de compilación que ejecuta `npm run build` durante la creación de la imagen y lanza la aplicación final optimizada mediante `next start -p ${PORT:-8501}`. Esto bajó el consumo de memoria a menos de la mitad y aceleró drásticamente la velocidad de carga de la página.

7.2. Base de datos PostgreSQL

Se utiliza PostgreSQL 15 como sistema de gestión de base de datos relacional. La interacción con la base de datos se realiza a través de SQLAlchemy ORM con la siguiente configuración:

- Motor de conexión con `client_encoding=utf8` para soporte de caracteres especiales en español.
- Sesiones gestionadas mediante un generador (`get_db`) que garantiza el cierre correcto de conexiones.
- Creación automática de tablas al iniciar la aplicación mediante `Base.metadata.create_all`.
- **Inicialización de datos (seeding):** Para simplificar las pruebas del sistema, programé una rutina de inicio en `main.py` llamada `seed_admin_user()`. Si el sistema detecta que la base de datos está vacía de perfiles administradores al arrancar, genera automáticamente una cuenta de administración por defecto (`admin@us.es` con la clave `adminpassword`) para poder probar el panel de control sin pasos previos de configuración.
- **Ordenación estable de mensajes:** Durante el testeo del chat con usuarios reales, detectamos un comportamiento molesto: a veces, al refrescar la página o enviar

mensajes rápidos, el orden cronológico de la conversación se alteraba en pantalla (mensajes del asistente aparecían antes que la pregunta correspondiente). La causa era que las consultas SQL de SQLAlchemy no garantizaban un orden predeterminado en las relaciones bidireccionales. Para resolverlo, añadí una regla explícita de ordenación en la declaración del modelo en `models.py`, forzando a que la relación cargue los mensajes ordenados secuencialmente mediante `order_by="Mensaje.id"`. Con este pequeño ajuste aseguramos la estabilidad del flujo visual.

Capítulo 8

Pipeline de ingesta de documentos

8.1. Procesamiento con LlamaParse

LlamaParse se configura con un `system_prompt` especializado en documentos burocráticos universitarios que instruye al modelo para:

1. Extraer todas las celdas de tablas sin omitir texto, incluyendo celdas combinadas.
2. Mantener notas al pie y aclaraciones inmediatamente después de la sección a la que hacen referencia.
3. Respetar la jerarquía de encabezados (H1, H2, H3) y listas enumeradas.
4. Extraer el texto íntegramente, sin resumir.

La extracción se realiza mediante el método `get_json_result`, que devuelve los resultados organizados por página, permitiendo asociar cada fragmento con su página de origen en los metadatos.

8.2. Fragmentación semántica (chunking)

8.2.1. Evolución de la estrategia de fragmentación

Si algo aprendí durante el desarrollo de este proyecto es que la forma de fragmentar los documentos importa muchísimo más de lo que parece a primera vista. De hecho, la estrategia de fragmentación evolucionó a lo largo de tres fases hasta dar con una solución satisfactoria:

1. **Fase inicial:** Fragmentación genérica con chunks de 1000 caracteres y solapamiento de 200. Los fragmentos carecían de contexto suficiente y las tablas quedaban rotas.

2. **Fase intermedia (Docling):** Se incorporó Docling para una extracción más inteligente. Mejoró la claridad del contexto pero seguía fallando con tablas y elementos complejos.
3. **Fase final (LlamaParse):** La migración a LlamaParse resolvió la preservación de tablas y jerarquías documentales, produciendo Markdown estructurado de alta calidad.

8.2.2. El problema del doble splitter

Este fue uno de esos errores sutiles que te vuelven loco. En un momento dado, las respuestas sobre baremos empezaron a degradarse sin razón aparente. Después de bastante tiempo depurando, descubrí que se estaban ejecutando dos procesos de fragmentación de forma redundante: LlamaParse ya producía fragmentos semánticos de calidad, pero añadí un `MarkdownTextSplitter` adicional que volvía a cortar esos fragmentos, rompiendo tablas y separando notas de su contexto.

La solución fue sencilla una vez identificado el problema: confiar en la segmentación de LlamaParse para la estructura semántica, y usar el `MarkdownTextSplitter` únicamente como red de seguridad para fragmentos que excedieran el tamaño máximo.

8.2.3. Configuración final: `MarkdownTextSplitter`

La configuración final del splitter utiliza:

- `chunk_size = 1500`: Tamaño suficiente para contener una sección completa con su contexto.
- `chunk_overlap = 250`: Solapamiento que garantiza continuidad entre fragmentos adyacentes.

Tras la fragmentación, se aplica `filter_complex_metadata` para limpiar metadatos incompatibles con Pinecone.

8.3. Generación de embeddings con Cohere

Se utiliza el modelo `embed-multilingual-v3.0` de Cohere para generar embeddings de 1024 dimensiones. Este modelo fue seleccionado por su rendimiento superior en textos en español frente a alternativas como los embeddings de HuggingFace, y por su capacidad de capturar matices semánticos en documentación normativa.

8.4. Almacenamiento en Pinecone

Los vectores se almacenan en un índice denominado `index-tfg` en Pinecone. La subida se realiza por lotes de 100 fragmentos para evitar timeouts y respetar los límites de la API. Cada vector almacena como metadata el nombre del documento fuente y el número de página, información que posteriormente se utiliza para generar las referencias en las respuestas.

Capítulo 9

Implementación del agente

9.1. Evolución de la estructura del agente (grafo de estados)

El diseño final del agente conversacional no surgió de manera inmediata; fue el resultado de un proceso iterativo de desarrollo y depuración a lo largo de varios sprints. Durante el diseño del flujo lógico en LangGraph, la arquitectura del grafo de estados evolucionó a través de tres fases preliminares antes de consolidar el diseño definitivo de ocho nodos que se utiliza actualmente.

9.1.1. Primera aproximación: Pipeline RAG lineal (Versión 1)

Al comenzar el desarrollo del agente, opté por la aproximación más directa y sencilla: un flujo lineal clásico de generación aumentada por recuperación. Esta primera versión constaba únicamente de dos nodos principales conectados en serie:

- **Nodo Recuperador:** Recibía la consulta directa del usuario, realizaba la conversión a embedding y recuperaba los fragmentos más relevantes de la base de datos vectorial en Pinecone.
- **Nodo Resultor (Generador):** Tomaba la pregunta original junto con los fragmentos recuperados y redactaba la respuesta final que se entregaba al usuario.

Esta estructura inicial (representada en la Figura 9.1) pecaba de ser extremadamente ingenua. Al no disponer de mecanismos de control, el sistema intentaba responder a cualquier tipo de entrada. Si un usuario saludaba o introducía una consulta completamente ajena al ámbito universitario (por ejemplo, preguntas de cultura general o de ocio), el recuperador igualmente forzaba una búsqueda en la base de datos de normativas de la Universidad de Sevilla. Esto no solo generaba un consumo absurdo de cuota de API, sino

que obligaba al modelo generador a alucinar o inventar respuestas para intentar casar textos legales con preguntas fuera de contexto.

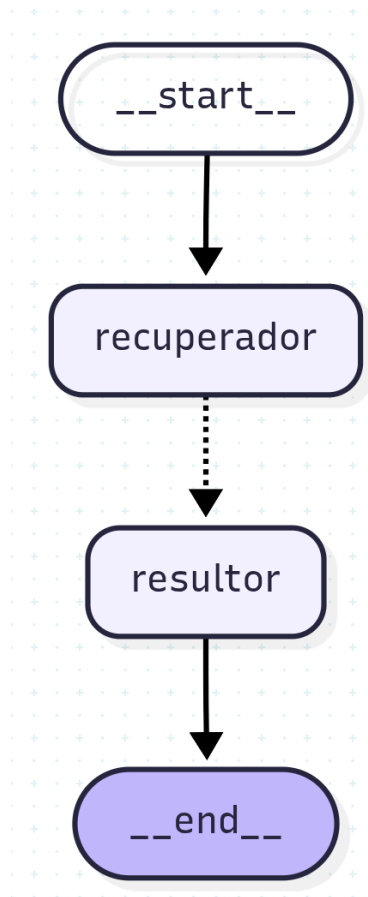


Figura 9.1: Primera versión del flujo del agente (RAG lineal básico)

9.1.2. Segunda aproximación: Incorporación de control y enrutamiento (Versión 2)

Para mitigar las limitaciones de la primera versión, implementé una estructura más sofisticada orientada a la gestión de intenciones. Introduje un enrutador lógico que permitía derivar la ejecución hacia tres nodos especializados según el tipo de interacción, aunque todos ellos seguían estando precedidos por el paso de recuperación:

- **Nodo Rechazo Amable:** Diseñado para declinar de forma educada y empática las consultas detectadas fuera de dominio.
- **Nodo Consultor (Entrevistador):** Encargado de solicitar aclaraciones y realizar preguntas de seguimiento en caso de detectar ambigüedades en la consulta del usuario.
- **Nodo Resultor (Generador):** Responsable de dar la respuesta final cuando la información era suficiente y pertinente.

En esta segunda iteración (ver Figura 9.2), el nodo recuperador se ejecutaba siempre al inicio del flujo, alimentando el estado compartido antes de que el clasificador decidiera el camino a tomar. Aunque esto solucionó el problema de las alucinaciones al permitir que el nodo de rechazo interceptara preguntas irrelevantes, introdujo una ineficiencia crítica de latencia y coste. Si el usuario enviaba un simple saludo (como "Hola") o una pregunta fuera de ámbito, el sistema realizaba de todos modos la llamada a la API de Cohere para generar el embedding y la consulta a Pinecone de manera innecesaria. El proceso completo tardaba varios segundos de más solo para acabar devolviendo un rechazo o un saludo básico.

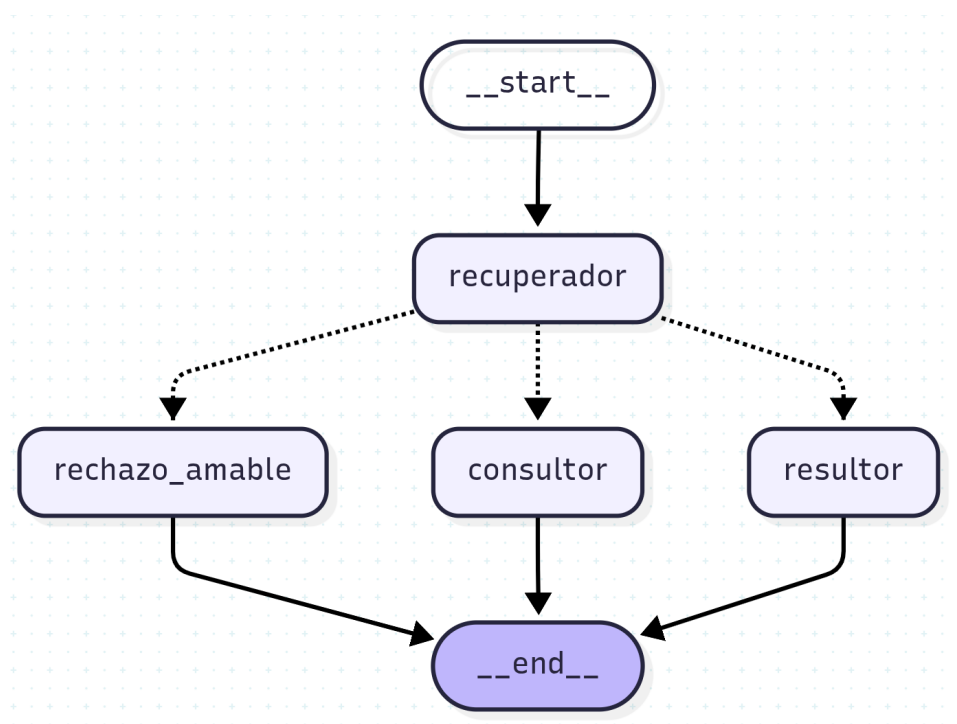


Figura 9.2: Segunda versión del flujo con enrutamiento posterior a la recuperación

9.1.3. Tercera aproximación: Recuperación condicional (Versión 3)

Para resolver el problema de rendimiento detectado en la segunda versión, reestructuré las conexiones del grafo de estados en una tercera versión que optimizaba el momento en el que se invocaba al recuperador.

En esta arquitectura (representada en la Figura 9.3), el flujo inicial ya no pasaba obligatoriamente por el recuperador. En su lugar, el sistema evaluaba en primer lugar si la consulta correspondía al dominio del agente. Si la entrada era ajena a la universidad, el flujo se desviaba de inmediato al nodo de **rechazo_amable**, terminando la ejecución de forma instantánea sin realizar ninguna llamada de embedding ni búsqueda en base de datos. El nodo recuperador se reubicó de manera condicional, situándose única y ex-

clusivamente en el camino hacia los nodos de **consultor** y **resultor**, que eran los que realmente requerían el contexto de la base de conocimiento normativa.

Esta versión supuso un salto de calidad enorme en cuanto a eficiencia y reducción de latencia en consultas básicas. Sin embargo, seguía teniendo un problema de precisión en el nodo **resultor**. Al haber un único nodo encargado de generar todas las respuestas correctas, su prompt del sistema se volvió extremadamente complejo y extenso. Tenía que contemplar directrices para redactar plazos temporales, explicar normativas académicas áridas, desglosar pasos administrativos de matrícula y construir tablas complejas para baremos de contratación. La contradicción interna entre estas instrucciones causaba que, a menudo, el modelo olvidase aplicar el formato adecuado (como usar tablas en baremos o listas numeradas en trámites).

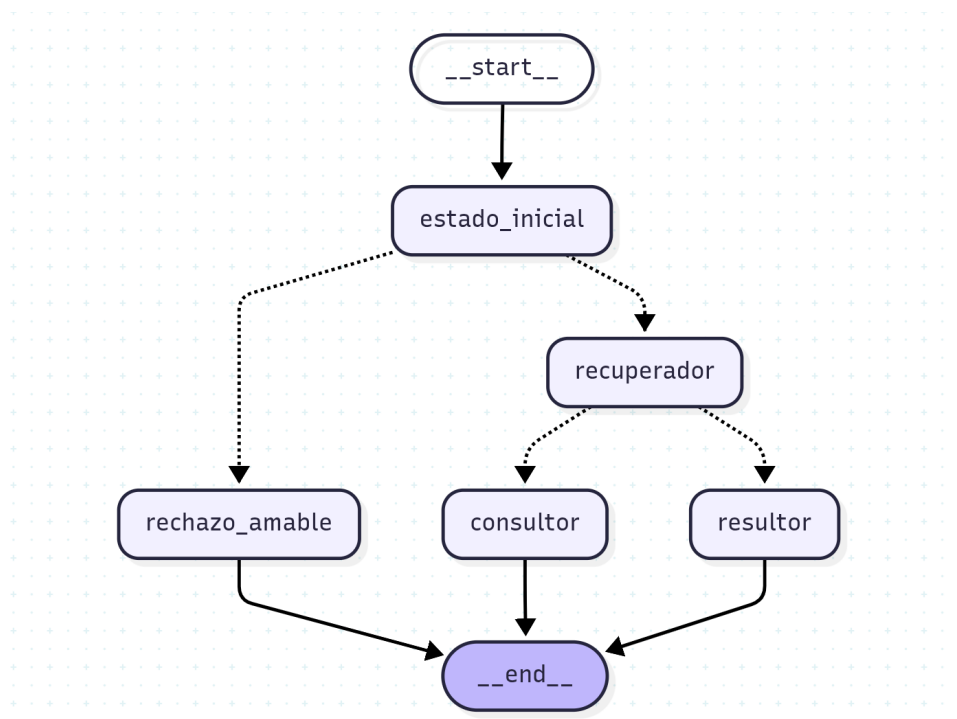


Figura 9.3: Tercera versión con recuperación condicionada a la relevancia de la consulta

Esta limitación fue el detonante para diseñar la **versión final del agente** detallada en el Capítulo 6 (ver Figura 6.2). En ella, eliminé el nodo **resultor** genérico y lo dividí en cuatro resolutores especializados por categorías (Procedimental, Calendario, Normativo y Baremo), precedidos por un clasificador de intenciones, lo que me permitió segmentar los prompts y asegurar un formato impecable para cada tipo de respuesta.

9.2. Esquema de estado (StateSchema)

El estado compartido entre todos los nodos del grafo se define mediante un `TypedDict` de Python con los siguientes campos:

- `pregunta (str)`: La consulta original del usuario.
- `pregunta_reformulada (str)`: La consulta enriquecida con contexto del historial.
- `historial (list)`: Lista de mensajes previos en formato `{role, content}`.
- `historial_formateado (list)`: Historial en formato texto plano para inyección en prompts.
- `contexto (str)`: Fragmentos de documentos recuperados del RAG.
- `stream (str)`: Generador de la respuesta para streaming.
- `referencias (list)`: Lista de fuentes consultadas.

9.3. Sistema de prompts

Todos los prompts del sistema se centralizan en un módulo `templates.py`, lo que facilita su mantenimiento y evolución iterativa.

9.3.1. Consolidación del Análisis Inicial con Salida Estructurada

En las primeras versiones del agente, la detección de la intención, la reformulación de la pregunta y la asignación de la categoría académica se ejecutaban en tres nodos distintos y consecutivos del grafo. Aunque el flujo lógico era correcto, esta secuencialidad castigaba severamente el rendimiento de la aplicación, pues obligaba a encadenar tres llamadas consecutivas al proveedor de LLM en la nube antes de poder siquiera buscar información en Pinecone.

Para resolver este cuello de botella, decidí unificar estas tres tareas en una sola llamada de red al inicio de la ejecución. Aprovechando las capacidades de salida estructurada de LangChain y Pydantic, definí un esquema estricto (`AnalisisInicial`) que le exige al modelo devolver un objeto JSON con tres campos bien definidos: `intencion`, `pregunta_reformulada` y `categoria`.

Mediante el método `with_structured_output()`, el LLM ligero procesa la consulta actual del usuario y el historial del chat en un único turno. Esta consolidación redujo el número de peticiones iniciales al LLM en un 40 % y aceleró notablemente la respuesta del sistema, garantizando además que los datos extraídos sigan un formato predecible sin riesgo de fallos en el tipado durante el enrutamiento.

9.3.2. Prompt de análisis inicial estructurado

Este prompt (`PROMPT_ANALISIS_INICIAL`) agrupa tres análisis críticos en una sola ejecución para optimizar la latencia: determina si la pregunta es relevante (*intención*),

reformula la consulta apoyándose en el historial de chat, y asigna la categoría académica de destino (*procedimental*, *calendario*, *normativo* o *baremo*). La salida se fuerza a cumplir el esquema Pydantic mencionado para garantizar robustez.

9.3.3. Prompt de suficiencia de información

Analiza si el contexto recuperado contiene información suficiente para dar una respuesta directa o si es necesario preguntar al usuario. Implementa una “Regla de Oro” que prohíbe devolver “entrevistador” si la última intervención del asistente fue una pregunta, evitando bucles de clarificación infinitos.

9.3.4. Prompts de los nodos resolutores

Cada nodo resolutor cuenta con un prompt especializado que define el formato, tono y nivel de detalle esperado. Los prompts incluyen instrucciones específicas sobre el uso de listas, tablas, negritas y emojis según la categoría académica seleccionada.

9.4. Pipeline de recuperación

9.4.1. Búsqueda Híbrida (Dense + Sparse) y RRF

En el diseño original, el sistema confiaba ciegamente en la búsqueda densa basada en embeddings vectoriales. Sin embargo, en un contexto burocrático universitario, esto presenta problemas. Las consultas que incluyen nombres específicos de trámites, siglas institucionales (como UVUS, automatrícula, baremación de contratos) o códigos de reglamentos a veces se diluyen en el espacio latente del modelo de embeddings, devolviendo fragmentos temáticamente cercanos pero sin el término exacto requerido.

Para solucionar esto, decidí implementar un motor de **Búsqueda Híbrida a nivel de aplicación**. Dado que el índice de Pinecone es denso y no quería duplicar ni reestructurar el índice entero en producción, opté por una estrategia de fusión:

1. **Recuperación densa:** Realizo una consulta vectorial en Pinecone utilizando la pregunta reformulada del usuario y el modelo de embeddings `embed-multilingual-v3.0` de Cohere, solicitando los 18 candidatos más cercanos ($k = 18$).
2. **Recuperación léxica (Sparse):** En paralelo, en el backend, realizo una búsqueda por palabra clave exacta usando una indexación y puntuación por frecuencia de términos (lexical search) sobre los fragmentos locales del documento. Esto puntúa los fragmentos que contienen los términos exactos de la consulta reformulada.
3. **Reciprocal Rank Fusion (RRF):** Una vez obtenidos ambos rankings de documentos candidatos, los fusiono en una lista única ponderando la posición que ocupa

cada fragmento en cada uno de los métodos. El cálculo matemático lo realizo mediante la siguiente ecuación:

$$RRF(d) = \frac{1}{60 + \text{Rank}_{\text{dense}}(d)} + \frac{1}{60 + \text{Rank}_{\text{sparse}}(d)}$$

donde el factor constante de regularización se establece en 60 para evitar que los primeros resultados monopolicen el peso de forma desproporcionada.

Los 12 fragmentos con mejor puntuación RRF los envío a la siguiente fase del pipeline.

9.4.2. Reranking y Compresión con Cohere

Los 12 mejores candidatos obtenidos tras la fusión RRF pasan por un filtro de reranking contextual con el modelo **rerank-v4.0-pro** de Cohere. Este modelo está especializado en calcular la relevancia directa de un fragmento de texto con respecto a una consulta en lenguaje natural.

El reranking actúa como un embudo crítico: reduce los 12 candidatos híbridos a los 4 documentos finales con mayor relevancia contextual (**top_n=4**). Esto disminuye la cantidad de tokens de ruido inyectados en la ventana de contexto del LLM generador, reduciendo costes y evitando la alucinación del modelo principal, además de asegurar que las tablas e instrucciones específicas de los reglamentos lleguen intactas.

9.5. Gestión de memoria conversacional

9.5.1. Ventana deslizante de mensajes

Para evitar exceder la ventana de contexto de los modelos de Groq, implementé una ventana deslizante que envía al agente únicamente los últimos 5 mensajes de la conversación. El historial completo se almacena en la base de datos, pero solo el subconjunto reciente se utiliza para la inferencia.

9.5.2. Compresión de memoria con resumen

Cuando una conversación supera los 5 mensajes, ejecuto una tarea en segundo plano (**BackgroundTasks** de FastAPI) que utiliza el LLM ligero para generar un resumen comprimido del historial antiguo. Este resumen lo almaceno en el campo **resumen_memoria** de la conversación y lo inyecto como mensaje de sistema al inicio del historial, preservando información clave (tipo de estudios, problema principal, fechas mencionadas) sin consumir tokens excesivos.

9.6. Generación automática de títulos

Al enviar el primer mensaje de una conversación con título “Nueva conversación”, disparo una tarea asíncrona que utiliza el LLM ligero para generar un título descriptivo de 4-5 palabras basado en el contenido del mensaje.

9.7. Corrective RAG (CRAG) y Búsqueda Web Fallback

El principal inconveniente de usar un RAG clásico es que si la información no está en los PDF de Pinecone (por ejemplo, plazos específicos de la automatrícula del curso actual que aún no se han publicado oficialmente en los reglamentos cargados), el asistente responde que no sabe nada o, peor aún, se inventa la respuesta.

Para resolver esto, implementé una arquitectura Corrective RAG (CRAG) en LangGraph. Justo después de recuperar los fragmentos, la arista condicional evalúa la relevancia y suficiencia de la información recuperada de Pinecone usando el modelo de lenguaje ligero, dividiendo el flujo en tres caminos posibles:

- **Suficiente (resultor):** Los fragmentos contienen la información necesaria para responder de forma directa. El flujo continúa hacia el clasificador de categorías y los nodos resolutores.
- **Ambiguo (entrevistador):** La consulta carece de parámetros clave para dar una respuesta definitiva. El flujo se deriva al nodo entrevistador para solicitar aclaraciones al usuario.
- **Insuficiente (busqueda_web):** Los fragmentos recuperados no contienen información útil para la consulta. El grafo transiciona al nodo de `busqueda_web` para realizar una búsqueda externa de fallback.

En el nodo de `busqueda_web`, realizo una consulta a la API de DuckDuckGo filtrando los resultados de forma estricta por la web de la US (`site:us.es`). Con esto logro que el agente se descargue fragmentos del portal oficial en tiempo real, los procese y los meta en el prompt del LLM principal para generar la respuesta definitiva con datos actualizados.

Como buscar en la web tarda un par de segundos, decidí enviar eventos intermedios al usuario mediante Server-Sent Events (SSE). De este modo, en la pantalla se muestra un estado dinámico (como “Buscando en el portal de la US...”) para que el usuario sepa qué está haciendo el agente en cada momento y no tenga la sensación de que la aplicación se ha quedado colgada.

9.8. Perfilado Dinámico y Memoria Persistente de Usuario

Tampoco quería que el asistente se sintiera frío o impersonal en conversaciones repetidas, así que desarrollé un sistema de perfilado dinámico que aprende del usuario de forma automática.

En la tabla de usuarios de PostgreSQL añadí una columna `perfil_metadata` tipo JSON. Al acabar cada mensaje del chat, en lugar de hacer esperar al usuario, lanzo una tarea asíncrona en segundo plano mediante `BackgroundTasks` de FastAPI:

1. Pasa el historial reciente por un LLM ligero para extraer datos clave del perfil.
2. Detecta información útil, como el rol del usuario (alumno, profesor, PAS), qué grado o máster estudia (por ejemplo, Medicina) o si le interesan ciertos trámites como las becas.
3. Guarda estos detalles en `perfil_metadata` de forma incremental, sin machacar la información que ya sabíamos del usuario.

La siguiente vez que el usuario abre un chat, el backend recupera estos metadatos y los inyecta en el prompt del sistema. De esta forma, si el asistente ya sabe que el usuario estudia Medicina y este le pregunta "¿cuándo empiezan las clases?", el RAG buscará el calendario específico de esa facultad directamente, ahorrándole al usuario tener que dar explicaciones extra en cada conversación.

9.8.1. Prevención de saturación del contexto en el perfilado

Al probar el sistema de perfilado de forma intensiva, descubrí un problema crítico de acumulación de tokens: el modelo de lenguaje ligero que extraía la información del perfil tendía a alucinar y almacenar en la estructura JSON del usuario un historial redundante de las consultas anteriores o transcripciones completas de los mensajes bajo claves personalizadas (como `preguntas_realizadas` o `conversaciones`). Esto provocaba que, a medida que el usuario charlaba más con el agente, el tamaño del perfil inyectado en el prompt creciera de forma desmedida, arriesgándose a saturar la ventana de contexto.

Para evitar esta explosión de tokens, implementé una doble directiva de sanitización en el script de perfilado (`profiling.py`):

- **Directivas negativas en el prompt:** Restringí estrictamente en las directrices del extractor la posibilidad de persistir fragmentos de texto conversacional, forzando valores atómicos y sencillos.

- **Limpieza programática de claves:** Programé una subrutina que limpia e intercepta activamente la respuesta del LLM antes de guardarla en la base de datos, eliminando de raíz cualquier propiedad que contenga palabras clave sospechosas o prohibidas (tales como `preguntas`, `respuestas`, `conversaciones` o `historial`). Con esto aseguramos la estabilidad de la memoria a largo plazo del sistema.

Capítulo 10

Implementación del frontend

10.1. Arquitectura del Cliente (Next.js y React)

Reconstruí por completo el frontend original basado en Streamlit como una aplicación modular bajo el ecosistema de **Next.js** y **React**, asegurando tiempos de carga inmediatos, una gestión de estado predecible y una interfaz moderna y fluida.

La estructura del código fuente la organicé de la siguiente manera:

- `src/app/page.tsx`: Componente y controlador raíz de la aplicación que maneja la verificación de sesión del usuario y el renderizado condicional de las vistas principales.
- `ChatInterface.tsx`: Componente principal para el chat interactivo, la visualización de flujos Server-Sent Events (SSE) y la recogida de feedback.
- `FeedbackList.tsx`: Vista administrativa de auditoría que recopila e ilustra el feedback negativo de los usuarios.
- `UserManagement.tsx`: Componente administrativo para la visualización, ascenso, degradación y borrado de cuentas de usuario.
- `DocumentList.tsx`: Panel administrativo que creé para ver de un vistazo qué normativas en PDF están subidas a Pinecone y poder eliminarlas del sistema si quedan obsoletas.
- `ConfirmModal.tsx`: Un modal de confirmación genérico y reutilizable que programé para evitar que se ejecuten acciones destructivas (como borrar un chat o eliminar un usuario) por un clic accidental.
- `src/utils/api.ts`: Módulo utilitario que abstrae la URL de conexión del backend.

10.2. Autenticación y Persistencia con JWT

Para la autenticación me apoyé en tokens JWT gestionados mediante cookies en el cliente (`js-cookie`). Al iniciar sesión con éxito en el endpoint `/login`, el frontend guarda el token del usuario y redirige al panel principal. Al cerrar sesión, la cookie se elimina, protegiendo las rutas privadas. Para las llamadas al backend, el utilitario `api.ts` añade automáticamente la cabecera `Authorization: Bearer <token>` a cada petición.

10.3. Interfaz del Chat y Consumo de SSE

El componente `ChatInterface.tsx` es el núcleo funcional para el usuario. Para evitar tiempos de espera aburridos, implementé el streaming real de respuestas consumiendo el endpoint `/conversaciones/{id}/chat` a través de la API del navegador `fetch` y un lector de flujos (`ReadableStream`).

El componente procesa dos tipos de eventos transmitidos por el backend:

1. **status:** Eventos intermedios del agente (p. ej., *"Pensando..."*, *"Buscando en el portal de la US..."*). El frontend captura este estado y lo renderiza de inmediato en una sección animada bajo la última burbuja, reduciendo sustancialmente la latencia percibida por el usuario.
2. **data:** Fragmentos de texto generados por el LLM en tiempo real. El cliente los concatena de inmediato al cuerpo del mensaje activo.

Para visualizar la respuesta final, utilizo el componente `ReactMarkdown`, mapeando elementos estándar a componentes personalizados de TailwindCSS para que las listas (`ul/ol`), los enlaces (`a`) y las negritas (`strong`) mantengan un estilo visual premium y uniforme.

10.3.1. Resolución dinámica de la conexión backend

Durante la puesta en producción en Render, el frontend experimentó desconexiones persistentes (`fetch failed` y errores de tipo 502 *Bad Gateway*). Inicialmente intenté configurar un proxy inverso mediante las directivas `rewrites` de Next.js en el servidor para enmascarar las llamadas al backend. Sin embargo, al tratarse de flujos continuos de eventos tipo SSE, el proxy de Next.js tendía a acumular el buffer de datos en memoria antes de mandarlo o daba fallos de resolución de DNS internos.

Para resolverlo de forma limpia y directa, eliminé el proxy intermedio y desarrollé un módulo utilitario de resolución en el cliente (`api.ts`). Este utilitario detecta en qué host se está ejecutando la aplicación desde el navegador: si es en desarrollo local, conecta directamente al puerto 8000; si detecta un dominio de producción, redirige todas las

peticiones a la URL pública y segura del backend en Render. Con este cambio eliminamos cualquier capa intermedia inestable y garantizamos la llegada fluida e instantánea del flujo SSE.

10.3.2. Tratamiento amigable de fallos y límites de API (SSE)

Uno de los principales problemas de los servicios en la nube gratuitos como Groq son sus estrictos límites de peticiones (errores de tipo 429 *Rate Limit Exceeded*). En las pruebas iniciales, cuando la API de Groq rechazaba una llamada por exceso de uso de tokens, el backend enviaba un evento crudo de error que provocaba que el chat del frontend se quedara congelado, sin dar feedback alguno al usuario de lo que estaba pasando.

Para mejorar la experiencia de usuario, desarrollé una función formateadora en el frontend llamada `formatSSEError()`. Esta función analiza de forma detallada el mensaje de error interceptado en la conexión SSE:

- **Límites de peticiones (429):** Si detecta un error de cuota del modelo, genera un mensaje con formato Markdown amigable explicando la situación. Además, mediante una expresión regular, extrae el tiempo de espera restante devuelto por Groq (por ejemplo, `try again in 1h32m26s`) y lo traduce a un formato legible por el usuario (indicando horas, minutos y segundos exactos) para que sepa exactamente cuándo puede volver a consultar al agente.
- **Exceso de contexto:** Si la conversación se ha vuelto excesivamente pesada y supera los límites del modelo, sugiere de manera elocuente iniciar un nuevo chat para limpiar la memoria activa.
- **Errores de red genéricos:** Si hay un problema de caída del servidor o falta de conexión, dibuja un aviso claro pidiendo que se revise la conexión de red, evitando cortes bruscos en el flujo de la aplicación.

10.4. Diseño del Feedback Loop en Interfaz

En cada mensaje del asistente, el frontend renderiza dos botones discretos para votar. Si el usuario hace clic en el botón que indica que la respuesta no es útil, la interfaz despliega inline una caja de texto con una animación de entrada suave para recoger su queja o comentario. Al hacer clic en enviar, el componente realiza una solicitud de tipo POST al backend registrando los campos de retroalimentación sin interrumpir el flujo del chat.

10.5. Panel de Auditoría y Administración

Los administradores tienen acceso a tres vistas avanzadas accesibles desde la barra lateral:

- **Gestión de Usuarios:** Una tabla interactiva para buscar cuentas registradas, promover usuarios a administradores o eliminar perfiles en cascada de la base de datos.
- **Auditoría de Feedback:** Renderizado por `FeedbackList.tsx`, presenta tarjetas de control con cada mensaje valorado con feedback negativo. Utiliza `ReactMarkdown` para mostrar la respuesta exacta generada por el agente y etiquetas visuales para ilustrar qué PDFs sirvieron como referencia en Pinecone y qué queja exacta dejó el usuario.
- **Ingesta de Documentos:** Área para subir nuevos PDFs de normativa, encolando el procesamiento asíncrono con `LlamaParse` mediante estados visuales de carga.

10.6. Implementación de Modo Claro y Oscuro Adaptativo

Para ofrecer una mejor accesibilidad visual, integré un sistema de modos de color dinámicos (modo claro y oscuro) mediante `TailwindCSS` bajo la estrategia de clase (`darkMode: 'class'`). Con esto logré que todos los componentes modifiquen sus colores y bordes de forma coherente mediante la presencia de la clase `dark` en la etiqueta raíz del documento (`<html>`).

La implementación técnica la estructuré en tres partes principales:

- **Variables CSS semánticas:** En el archivo `globals.css` definí los tokens de color del sistema de diseño (tales como `--color-background`, `--color-sidebar`, `--color-text-main`, `--color-border`, etc.) para ambos esquemas cromáticos, facilitando que `Tailwind` los referencee de manera dinámica sin necesidad de duplicar clases condicionales en el código.
- **Detección y persistencia de preferencias:** En el componente raíz `page.tsx`, mediante un *hook* de efecto (`useEffect`), programé la aplicación para que consulte en primer lugar si existe una preferencia previa guardada en el almacenamiento del navegador (`localStorage`). Si no existe, recurre a la consulta de medios (`prefers-color-scheme: dark`) mediante el método `window.matchMedia()` para alinear el tema de inicio con la configuración del sistema operativo del dispositivo del cliente.

- **Control de alternancia interactivo:** En la barra lateral (`Sidebar.tsx`), junto a los botones de navegación y cierre de sesión, añadí un botón con iconos adaptativos (`Sun` y `Moon` de `lucide-react`) para alternar el tema en tiempo de ejecución. Al pulsarse, se actualiza el estado de la aplicación, guardo la nueva configuración en `localStorage` y añado o elimino la clase `dark` en el objeto raíz `document.documentElement`.

Capítulo 11

Selección y configuración de modelos

11.1. Evolución de la infraestructura de inferencia

11.1.1. Fase 1: Modelos locales (Llama 3.1 8B)

Al principio del proyecto, la idea era ejecutar todo en local usando Llama 3.1 8B a través de Ollama. El modelo cumplía bastante bien con las tareas de clasificación, pero los tiempos de inferencia eran completamente inaceptables: entre 30 y 60 segundos por cada llamada al modelo. Para un asistente que se supone que debe responder en tiempo real, esto era inviable. Además, la ejecución local hacía imposible cualquier tipo de despliegue más allá de mi propio ordenador.

11.1.2. Fase 2: API de HuggingFace

El siguiente paso fue migrar a los endpoints de inferencia de HuggingFace, lo que mejoró bastante los tiempos. Pero apareció otro problema que no había previsto: el *cold start*. La primera solicitud tras un rato de inactividad podía tardar hasta 30 segundos mientras la instancia se aprovisionaba. Era frustrante porque funcionaba bien en las pruebas seguidas, pero en cuanto dejabas de usar el sistema un rato, la primera respuesta tardaba una eternidad.

11.1.3. Fase 3: API de Groq

La solución definitiva llegó con la API de Groq, que utiliza un hardware especializado llamado LPU (*Language Processing Units*) diseñado específicamente para ejecutar modelos de lenguaje. El cambio fue drástico: los tiempos de respuesta bajaron a fracciones de segundo por invocación, sin ningún problema de inicio en frío. La única pega es que la capa gratuita tiene límites de tokens por minuto y por día, pero para el ámbito de este proyecto fue más que suficiente.

11.2. Estrategia multi-modelo

Para optimizar el consumo de tokens y la latencia, diseñé e implementé una estrategia de tres niveles de modelo:

11.2.1. LLM clasificador (Llama 3.1 8B Instant)

Configurado con `temperature=0.0` y `max_tokens=15`. Se utiliza exclusivamente para tareas de clasificación binaria (detección de intención, suficiencia de información, categorización). Su respuesta es una única palabra clave, lo que minimiza el consumo de tokens.

11.2.2. LLM ligero (Llama 3.1 8B Instant)

Configurado con `temperature=0.1` y `max_tokens=300`. Se emplea para reformulación de consultas, generación de títulos, compresión de memoria y respuestas del nodo entrevistador y rechazo amable. Al tratarse de un modelo de 8B parámetros, ofrece baja latencia con calidad suficiente para estas tareas.

11.2.3. LLM principal (Llama 3.3 70B Versatile)

Configurado con `temperature=0.1` y `max_tokens=1500`. Se reserva para los nodos resolutores, donde la calidad de la respuesta es crítica. El modelo de 70B parámetros demuestra una capacidad cognitiva superior para interpretar normativa compleja y sintetizar información de múltiples fragmentos.

11.3. Ajuste de hiperparámetros

De todos los parámetros con los que experimenté, la `temperature` fue sin duda el más determinante. Con valores altos (0.7 o más), el modelo alucinaba con frecuencia preocupante; con 0.0, las respuestas eran correctas pero sonaban demasiado mecánicas. Al final me quedé con 0.1, que era el punto justo en el que el agente priorizaba la precisión factual pero seguía siendo capaz de parafrasear y adaptar su tono según el contexto. También configuré `max_retries` a 10 para el modelo principal, como colchón ante los posibles errores de *rate-limiting* de la API de Groq.

Parte V

Evaluación y Validación

Capítulo 12

Estrategia de evaluación

12.1. Dataset de evaluación

Para poder evaluar el sistema de forma rigurosa, diseñé un dataset con 50 preguntas que cubren todos los nodos del agente. La idea era simular consultas reales que podría hacer un estudiante, un profesor o un candidato a una plaza. Las preguntas se distribuyeron de la siguiente manera:

Tabla 12.1: Distribución del dataset de evaluación por categoría

Categoría	N.º de preguntas
Calendario	8
Procedimental	8
Normativo	8
Baremo	10
Consulta (ambigua)	8
Rechazo	8
Total	50

Cada caso de prueba incluye: la pregunta del usuario, la ruta esperada del router y un *ground truth* que describe la respuesta correcta esperada.

12.2. Métricas empleadas

12.2.1. Routing Accuracy

Mide el porcentaje de consultas en las que el agente clasifica correctamente la intención del usuario y la enruta al nodo especializado adecuado. Es una métrica binaria: acierto (1) o fallo (0) por cada caso.

12.2.2. Faithfulness

Evalúa en una escala de 1 a 5 si la respuesta generada se basa exclusivamente en la información presente en el contexto recuperado, sin inventar datos. Una puntuación de 5 indica que toda la respuesta está fielmente soportada por los documentos.

12.2.3. Answer Relevancy

Evalúa en una escala de 1 a 5 si la respuesta del agente aborda adecuadamente la pregunta del usuario. Una puntuación de 5 indica que la respuesta responde perfectamente y de forma completa a lo preguntado.

12.2.4. Correctness

Compara la respuesta generada contra el *ground truth* definido, evaluando en una escala de 1 a 5 la coincidencia factual entre ambas. Una puntuación de 5 indica concordancia plena.

12.3. Evaluación con LLM como juez

Se adoptó el paradigma *LLM-as-Judge*, utilizando Llama 3.1 ejecutado localmente mediante Ollama como evaluador automático. Este enfoque evita el consumo de tokens de APIs externas durante la fase de evaluación.

El proceso de evaluación para cada caso de prueba sigue este pipeline:

1. El agente procesa la pregunta recorriendo el grafo completo (detección → recuperación → clasificación → generación).
2. Se registra la ruta tomada y se compara con la esperada (Routing Accuracy).
3. El LLM juez evalúa la respuesta generada contra el contexto (Faithfulness), contra la pregunta (Relevancy) y contra el ground truth (Correctness).
4. Los resultados se exportan a un archivo CSV para análisis posterior.

12.4. Observabilidad y Monitoreo con Langfuse

Para un desarrollo profesional, no basta con evaluar el sistema en local; es necesario monitorizar el comportamiento del agente en producción, registrando cada llamada a las APIs, la latencia de cada nodo y los costes de inferencia.

Para cubrir esta necesidad, integré **Langfuse**, una plataforma de observabilidad de código abierto específica para aplicaciones basadas en LLMs. La integración en el backend

la realicé de forma limpia mediante el callback handler oficial de Langfuse para LangChain. Cuando el agente LangGraph procesa una consulta:

- Se registra la traza completa de la ejecución en el backend.
- Se mide con precisión el tiempo de respuesta (latencia) de cada nodo (enrutador, recuperador, clasificadores y resolutores).
- Se monitoriza el número de tokens consumidos en cada llamada a la API de Groq, lo que permite evaluar el coste económico en tiempo real.
- Permite auditar el comportamiento del agente y depurar de forma interactiva en la consola de Langfuse si ocurre algún comportamiento inesperado o alucinación.

12.5. Evaluación científica offline con Ragas

Además de la monitorización en tiempo real, implementé un script de evaluación offline automatizado en `evaluar_rag.py` basado en el framework **Ragas**. Este framework evalúa de manera científica la calidad del pipeline RAG midiendo dos métricas esenciales utilizando un LLM como juez independiente (`llama-3.3-70b-versatile`):

1. **Faithfulness (Fidelidad):** Mide el porcentaje de afirmaciones en la respuesta que se pueden contrastar directamente con el contexto recuperado de los documentos de Pinecone, sirviendo como detector objetivo de alucinaciones.
2. **Answer Relevance (Relevancia de la respuesta):** Evalúa si la respuesta aborda directamente la pregunta formulada por el usuario o si introduce información irrelevante que no responde a su duda original.

El script ejecuta un conjunto controlado de casos de prueba reales sobre la normativa de la US, compara las salidas con el ground truth definido y almacena las calificaciones en un informe formateado en CSV. Esto facilita el análisis estadístico directo de los datos experimentales para el TFG.

Parte VI

Conclusiones y Trabajo Futuro

Capítulo 13

Conclusiones

13.1. Cumplimiento de objetivos

Mirando hacia atrás, creo que el proyecto ha cumplido de forma satisfactoria los objetivos que se plantearon al inicio:

- **Ingesta semántica:** Desarrollé un pipeline capaz de procesar documentos oficiales de la US, preservando tablas, jerarquías y notas al pie mediante LlamaParse.
- **Enrutamiento inteligente:** El agente LangGraph clasifica correctamente las consultas y las dirige al nodo especializado correspondiente, con una alta precisión de enrutamiento.
- **Sistema Dual RAG:** Implementé la separación entre conocimiento normativo (Pinecone) y directrices de estilo (prompts), logrando respuestas técnicamente correctas con un tono cercano al usuario.
- **Latencia optimizada:** La migración a Groq y la estrategia multi-modelo redujeron los tiempos de respuesta por debajo del umbral de 20 segundos establecido.
- **Validación:** Evalué el sistema con 50 casos de prueba utilizando el paradigma LLM-as-Judge.

13.2. Lecciones aprendidas

Más allá del resultado técnico, hay varias lecciones que me llevo de este trabajo y que creo que merece la pena compartir:

- **La ingesta importa más que el modelo:** Perdí bastante tiempo al principio intentando mejorar las respuestas cambiando de modelo, cuando el verdadero problema estaba en cómo fragmentaba los documentos. Una mala segmentación invalida cualquier mejora que hagas aguas abajo.

- **La temperatura es el hiperparámetro más crítico:** Parece un detalle menor, pero pasar de 0.7 a 0.1 fue la diferencia entre un agente que alucinaba con frecuencia y uno que se ceñía a los documentos. El valor de 0.1 resultó ser el punto justo entre precisión factual y cierta naturalidad en la redacción.
- **Los prompts son código:** La ingeniería de prompts requiere el mismo rigor iterativo que el desarrollo de software. Pequeños cambios en una palabra pueden alterar por completo el comportamiento del agente.
- **No todo necesita el modelo grande:** Usar Llama 3.3 70B para tareas de clasificación binaria era como matar moscas a cañonazos. La estrategia multi-modelo no solo reduce costes, sino que también baja la latencia de forma significativa.

13.3. Dificultades encontradas

No todo fue un camino de rosas, ni mucho menos. Estas fueron las dificultades más relevantes:

- **Procesamiento de tablas en PDFs:** Sin duda, el mayor dolor de cabeza técnico. Hicieron falta tres intentos distintos (fragmentación genérica → Docling → Llama-Parse) hasta conseguir que las tablas se preservaran correctamente.
- **Límites de APIs gratuitas:** Trabajar con las capas gratuitas de Groq, Cohere y LlamaParse fue un ejercicio constante de optimización. Cada token contaba, y hubo días en los que agoté la cuota de la API a mitad de una sesión de pruebas.
- **Bucles de clarificación:** Al principio, el nodo entrevistador tendía a hacer preguntas una y otra vez sin avanzar. Se resolvió implementando lo que llamé la “Regla de Oro” en el prompt de suficiencia: si el asistente acaba de preguntar, no puede volver a preguntar en el siguiente turno.
- **Redundancia del doble splitter:** Un error sutil que estuvo degradando las respuestas sobre baremos durante semanas hasta que lo localicé analizando el pipeline completo paso a paso.
- **Errores de serialización en telemetría (Generadores):** Durante la integración de la observabilidad de Langfuse con LangGraph, la serialización del estado del grafo fallaba con la excepción `TypeError: cannot create 'generator' instances`. Esto ocurría porque Pydantic v1 intentaba iterar y consumir de forma prematura los generadores de streaming de tokens (`stream`) para enviarlos a Langfuse. Lo solucioné encapsulando el generador en un contenedor no iterable personalizado (`StreamWrapper`), logrando registrar las trazas y cediendo el consumo del flujo de tokens de forma segura a la interfaz web.

- **Truncado de respuestas en consultas de aclaración:** Al realizar consultas muy amplias o ambiguas (como preguntas sobre becas), las explicaciones preliminares del entrevistador saturaban el presupuesto de 300 tokens asignado al modelo ligero. Esto provocaba que la API de Groq cortara la respuesta antes de formular la pregunta aclaratoria al usuario. Solventé esta limitación elevando los tokens máximos del modelo ligero a 1000 tokens, garantizando que el usuario reciba siempre la respuesta completa.
- **Falta de contexto fresco en consultas ambiguas:** Cuando el usuario hacía una consulta ambigua, el nodo entrevistador dependía únicamente de la recuperación inicial de Pinecone. Si la ambigüedad impedía obtener fragmentos útiles, el entrevistador carecía de información para plantear opciones claras. Para solucionarlo, doté al entrevistador de la herramienta de búsqueda web complementaria `buscar_web_us`, permitiéndole enriquecer el contexto en tiempo de ejecución con resultados del portal oficial de la US antes de formular la aclaración.

Capítulo 14

Trabajo futuro

14.1. Mejoras en el pipeline RAG

Una de las líneas que más me gustaría explorar es la implementación de *Hypothetical Document Embeddings* (HyDE). La idea consiste en generar un documento hipotético a partir de la pregunta del usuario antes de lanzar la búsqueda en la base vectorial, lo que en teoría mejoraría bastante la precisión semántica de la recuperación.

14.2. Ampliación de la base de conocimiento

El sistema está pensado para escalar sin demasiado esfuerzo. La funcionalidad de ingesta desde el panel de administración ya permite añadir nuevos documentos sin tocar una línea de código. Una extensión natural sería incorporar documentos adicionales como planes de estudio, guías docentes o la normativa específica de TFG.

14.3. Integración con canales de mensajería

Otra vía que creo que tiene mucho potencial es conectar el agente con plataformas de mensajería como Telegram o WhatsApp. Al fin y al cabo, si el objetivo es que el usuario pueda resolver sus dudas de la forma más cómoda posible, tiene todo el sentido que pueda hacerlo directamente desde la aplicación que ya usa a diario.

14.4. Despliegue en producción

Para llevar este sistema a un entorno real, la idea sería migrarlo a un servicio cloud (AWS, Google Cloud o Azure) con balanceo de carga y escalado automático. También sería necesario añadir monitorización de métricas en tiempo real, cerrando así el ciclo de mejora continua.

Bibliografía

- [1] Amazon Web Services. ¿Qué es un modelo de lenguaje grande (LLM)? Disponible en: <https://aws.amazon.com/es/what-is/large-language-model/>
- [2] LangChain AI. LangChain Documentation. Disponible en: <https://python.langchain.com/>
- [3] LangGraph Documentation. Disponible en: <https://langchain-ai.github.io/langgraph/>
- [4] RAGAS Framework. Automated Evaluation of RAG. Disponible en: <https://docs.ragas.io/>
- [5] Pinecone. Vector Database Documentation. Disponible en: <https://docs.pinecone.io/>
- [6] Cohere. Embeddings and Reranking API. Disponible en: <https://docs.cohere.com/>
- [7] Groq. LPU Inference Engine. Disponible en: <https://groq.com/>
- [8] LlamaIndex. LlamaParse Documentation. Disponible en: <https://docs.llamaindex.ai/>
- [9] FastAPI. Modern web framework for building APIs. Disponible en: <https://fastapi.tiangolo.com/>
- [10] React Library. Documentation for building user interfaces. Disponible en: <https://react.dev/>
- [11] Docker. Container platform documentation. Disponible en: <https://docs.docker.com/>
- [12] Vaswani, A. et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*.
- [13] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

- [14] Meta AI. Llama 3: Open Foundation Language Models. Disponible en: <https://ai.meta.com/llama/>

Apéndice A

Prompts del sistema

Para asegurar la reproducibilidad de los resultados, se adjuntan en este anexo los prompts exactos (plantillas de sistema) utilizados en los distintos nodos de LangGraph.

```
PROMPT_ANALISIS_INICIAL = """
```

```
    Analiza la conversacion actual y la ultima pregunta del
    usuario en el contexto de la Universidad de Sevilla (US).
```

```
    Tu tarea es realizar tres tareas en una sola llamada:
```

```
    1. INTENCION: Determina si el usuario desea hacer una
    consulta academica o tramite administrativo relacionado con la
    US (matricula, becas, plazos, normativas, expedientes,
    liquidacion de viajes de investigacion, etc.).
```

```
        - Devuelve 'rechazo_amable' si la consulta no tiene
    relacion con el ambito universitario o burocratico de la US (ej
    : recetas de cocina, ocio, programacion general, chistes,
    deportes o torneos internacionales como la Champions, Europa
    League, La Liga, etc.).
```

```
        - Devuelve 'recuperador' si esta relacionada con la
    universidad. En caso de duda razonable sobre si es un tema
    universitario, prioriza 'recuperador'.
```

```
    2. PREGUNTA REFORMULADA: Reformula la ultima pregunta del
    usuario para que sea una pregunta independiente y clara,
    incorporando el contexto del historial de chat. Si la pregunta
    actual es independiente y ya es clara por si sola, conservala
    igual.
```

```
    3. CATEGORIA: Clasifica la consulta academica del usuario
    segun el tipo de respuesta que requiere:
```

```
        - 'procedimental': Si el usuario necesita
```

instrucciones paso a paso para completar un tramite burocratico
.

- 'calendario': Si el usuario pregunta por fechas, plazos o periodos del calendario academico.

- 'normativo': Si el usuario pregunta por reglas generales, derechos, articulos de reglamento, normativas o informacion general.

- 'baremo': Si el usuario pregunta por criterios de evaluacion, meritos o calculo de puntuaciones en contrataciones u oposiciones.

- 'ninguna': Si la consulta no esta relacionada con la universidad (intencion 'rechazo_amable').

Historial de conversacion:

{historial}

Pregunta actual del usuario:

{question}

"""

PROMPT_CUESTIONA_AGENTE = """

Eres un experto consultor de la Universidad de Sevilla.

Tu único objetivo es decidir si el flujo debe ir a "entrevistador" o a "resultor".

REGLA DE ORO ESTRICTA Y ABSOLUTA:

Analiza el final del "Historial de conversación". Si la ÚLTIMA intervención del Asistente fue una pregunta pidiendo más datos (ej: terminaba con signo de interrogación), ESTÁ Estrictamente Prohibido Devolver "entrevistador". Debes devolver OBLIGATORIAMENTE "resultor", independientemente de si la respuesta del usuario está incompleta o no. ¡PROHIBIDO HACER DOS PREGUNTAS SEGUIDAS AL USUARIO!

CRITERIOS PARA "entrevistador":

- Es una consulta NUEVA (o el asistente no ha preguntado nada justo antes), el contexto indica explícitamente que falta un dato CLAVE para aplicar distintas reglas, y no es posible dar la información de las dos opciones a la vez.

CRITERIOS PARA "resultor":

- El usuario acaba de responder a una pregunta tuya (aplica la Regla de Oro).
- O el contexto tiene información suficiente para dar la respuesta directa.
- O se pueden explicar todos los casos posibles ("Si tu caso es X pasa esto, si es Y pasa lo otro").
- O EN CASO DE DUDA entre las dos opciones.

TU RESPUESTA DEBE SER ÚNICAMENTE UNA DE ESTAS DOS PALABRAS (en minúsculas, sin puntos ni texto extra):

entrevistador
resultor

Historial de conversación:
{historial}

Contexto Recuperado:
{context}

Pregunta del usuario:
{question}

Salida:
"""

PROMPT_REFORMULACION = """

Dada la siguiente conversación y la última intervención/ pregunta del usuario, reformula su intervención

para que sea una pregunta/afirmación independiente que contenga todo el contexto (sujetos, trámites, datos previos aclarados, etc.).

Es CRÍTICO que la pregunta reformulada mantenga todos los datos y aclaraciones que el usuario ha dado en el historial.

NO respondas a la pregunta, SOLO devuelve la pregunta reformulada. Si ya es clara por sí sola, devuélvela tal cual integrando sus respuestas.

Historial de conversación:
{historial}

Pregunta del usuario: {question}

Pregunta reformulada:
"""

PROMPT_CONSULTA_USUARIO = """

Eres el Asistente Experto de Atención al Estudiante y Soporte de la Universidad de Sevilla.

El sistema ha detectado que la consulta del usuario es ambigua o le faltan datos para darle la resolución definitiva según la normativa.

INSTRUCCIONES CLAVE Y ORDEN DE RESPUESTA:

1. RESPUESTA GENERAL (OBLIGATORIA): Basándote en el contexto recuperado, debes ofrecer SIEMPRE primero una respuesta informativa y cordial que oriente al usuario explicando las opciones generales que contempla la normativa.

2. PREGUNTA ACLARATORIA (AL FINAL): Justo después de tu explicación general, haz UNA ÚNICA pregunta directa para obtener el dato exacto que te falta (ej: si es grado o máster, si es primera matrícula, etc.).

REGLAS ESTRUCTURADAS:

- Asume que el usuario es de la US. NUNCA preguntes por su vinculación.

- NO inventes normativas ni supongas datos que no están en el contexto.

- Revisa el historial: NO vuelvas a preguntar un dato que el usuario ya haya proporcionado.

HISTORIAL DE CONVERSACIÓN:

{historial}

CONTEXTO NORMATIVO RECUPERADO:

{context}

PREGUNTA ACTUAL DEL USUARIO:

{question}

TU RESPUESTA (Explicación general primero + Pregunta aclaratoria al final):

"""

PROMPT_RECHAZO_AMABLE = ""

Eres un Asistente de Atención al Estudiante y Soporte de la Universidad de Sevilla.

Tu objetivo en este momento es indicar amablemente al usuario que no puedes ayudarlo con su consulta, porque no es una duda burocrática o no se puede resolver con la información del contexto.

INSTRUCCIONES CLAVE:

1. DIAGNÓSTICO: Si la pregunta del usuario no tiene relación con temas burocráticos universitarios, o si claramente no se puede responder con la información del contexto (ejemplo: preguntas sobre eventos culturales, vida en el campus, etc.), debes indicar amablemente que no puedes ayudar con esa consulta.

2. EXPLICACIÓN: Explica brevemente por qué no puedes ayudarlo (ejemplo: "Lamento no poder ayudarte con esa consulta, ya que no está relacionada con trámites o normativas universitarias...").

3. SUGERENCIA: Si es posible, sugiere al usuario dónde puede encontrar más información o a quién puede dirigirse para su consulta (ejemplo: "Te recomendaría contactar con el departamento de vida universitaria para este tipo de dudas...").

4. TONO: Educado, empático y resolutivo. NO inventes información ni trates de responder a la consulta si no es una duda burocrática o no se puede resolver con el contexto.

HISTORIAL DE CONVERSACIÓN:

{historial}

CONTEXTO NORMATIVO RECUPERADO:

{context}

PREGUNTA ACTUAL DEL USUARIO:

{question}

RESPUESTA DEL ASISTENTE:

""

```
PROMPT_RESULTOR_PROCEDIMENTAL = ""
```

Eres un asistente de la Universidad de Sevilla (US) especializado en guiar a los usuarios a través de trámites administrativos (matrículas, viajes, etc.).

Basándote en el contexto proporcionado, explica de forma CLARA, ESTRUCTURADA y PASO A PASO cómo realizar el trámite.

INSTRUCCIONES:

1. Utiliza listas numeradas (1., 2., 3.) para los pasos secuenciales.
2. Resalta en ****negrita**** los nombres de plataformas web (ej. SEVIUS, Secretaría Virtual), nombres de impresos y documentos requeridos.
3. Si hay advertencias importantes o requisitos previos, ponlos al principio bajo un encabezado "[!] Requisitos Previos".

Historial de conversación:

{historial}

Contexto recuperado:

{context}

Pregunta del usuario sobre el procedimiento:

{question}

Instrucciones paso a paso:

""

```
PROMPT_RESULTOR_CAENDARIO = ""
```

Eres un asistente de la Universidad de Sevilla (US) especializado en el calendario académico y administrativo.

Basándote en el contexto proporcionado, responde a la pregunta del usuario prestando especial atención a las FECHAS, PLAZOS y PERIODOS.

INSTRUCCIONES:

1. PRECISIÓN: Sé extremadamente preciso con los días,

meses y años. No inventes ninguna fecha que no esté en el contexto. Si el contexto no especifica el año, indícalo.

2. CLARIDAD: Resalta las fechas clave en ****negrita**** para que sean fáciles de localizar.

3. FORMATO LIBRE: Elige el formato que mejor se adapte a la respuesta (un párrafo, una lista con viñetas, o una tabla si realmente hay muchos datos tabulares). No fuerces tablas cuando una lista o un párrafo es más claro.

4. CONTEXTO ADICIONAL: Si hay condiciones especiales, plazos extraordinarios o excepciones, menciónalas brevemente.

Historial de conversación:

{historial}

Contexto recuperado:

{context}

Pregunta del usuario sobre plazos/fechas:

{question}

Respuesta:

""

PROMPT_RESULTOR_NORMATIVO = ""

Eres un asistente legal y normativo de la Universidad de Sevilla (US).

Basándote en el contexto proporcionado, explica la normativa aplicable a la duda del usuario de forma comprensible, pero manteniendo el rigor formal.

Si el contexto menciona artículos específicos, normativas concretas o resoluciones rectorales, cítalos en tu respuesta para dar validez a la información.

Historial de conversación:

{historial}

Contexto:

{context}

Pregunta del usuario sobre la normativa:
{question}

Explicación normativa:
"""

PROMPT_RESULTOR_BAREMO = """

Eres un asistente de la Universidad de Sevilla (US)
experto en procesos de evaluación, baremación de méritos y
contratación.

Basándote en el contexto proporcionado, detalla cómo se
calculan los puntos, cuáles son los criterios de evaluación o
los requisitos mínimos.

INSTRUCCIONES DE FORMATO:

1. Utiliza una tabla Markdown para mostrar los criterios
de evaluación siempre que sea posible. Columnas sugeridas: "
Criterio" | "Detalles".
2. Desglosa los apartados puntuables claramente.
3. Si existen requisitos mínimos excluyentes, menciónalas
en una lista con viñetas al principio de tu respuesta.

Historial de conversación:
{historial}

Contexto:
{context}

Pregunta del usuario sobre baremación/evaluación:
{question}

Criterios de evaluación y puntuación:
"""

Apéndice B

Uso de la Inteligencia Artificial

En cumplimiento con las directrices académicas, se declara el uso de modelos de lenguaje para la asistencia en la generación de código LaTeX y el refactor de scripts de evaluación.